Undergraduate Hardware Projects

Sayeed Hossain Sayor

B.Sc. in Engineering (EEE)
First Class (Hons.)

Islamic University of Technology (IUT)
Organisation of Islamic Cooperation (OIC)
Department of Electrical and Electronic Engineering

#	Project	Course	Year
01	**SAP-1 Computer Architecture**	EEE 4308 — Digital Electronics Lab	2023
02	**Project Nema — Automated Aquarium**	EEE 4518 — Digital Electronics Lab	2023
03	**Digital Alarm Clock**	EEE 4706 — Microcontroller & System Design Lab	2023
04	**High Voltage Taser**	EEE 4504 — Power Electronics	2023

Table of Contents

Project 1 — SAP-1 Computer Architecture

1. **Introduction to SAP Computers**
2. **Architecture of SAP-1**
3. **The 8-bit Bus**
4. **Program Counter (PC)**
5. **Memory Address Register (MAR)**
6. **Input Unit**
7. **RUN/PROG Selector Switch**
8. **Random Access Memory (RAM)**
9. **Arithmetic and Logic Unit (ALU)**
10. **Output Unit**
11. **Instruction Detail Reference**
    - 11.1 Instruction: SUB (Subtract)
    - 11.2 Instruction: OUT (Output)
    - 11.3 Instruction: HLT (Halt)
12. **Complete Control Signals Summary**
13. **Instruction Cycle and T States**
14. **Fetch and Execution Cycles**
    - 14.1 Fetch Cycle (T1, T2, T3)
    - 14.2 Execution Cycle (T4, T5, T6)
    - 14.3 Complete T-State Summary
15. **Example Program**
16. **Hardware Implementation (PCB)**
    - 16.1–16.9 Individual PCB Modules

Project 2 — Project Nema: Automated Aquarium

Abstract

1. **Introduction**
2. **Overview of Components**
    - 2.1 Principal Components
    - 2.2 Basic Components
3. **System Operation Overview**
    - 3.1 Methodology Flow
    - 3.2 Circuit Diagrams
    - 3.3 Functionality Analysis
4. **Features**
5. **Implementation**
    - 5.1 Hardware Implementation
    - 5.3 Arduino Pin Connections

6. **Evaluation**
7. **Future Enhancements**
8. **Challenges**
9. **Estimated Cost**
10. **Conclusion**

Project 3 — Digital Alarm Clock

1. **Objective**
2. **Required Components**
3. **Circuit Diagram**
    3.3 Port & Pin Assignment
4. **Features**
5. **System Flowchart**
6. **Working Principle**
    6.1 Initialization Phase
    6.2 Main Loop Operation
    6.3 Mode Switching
    6.4 Timer 1 Interrupt (TIMER1ISR)
    6.5 Keypad Scanning & Input Handling
7. **Hardware Implementation**
8. **Problems Faced**
    8.1 Software Issues
    8.2 Hardware Issues
9. **Conclusion**

Project 4 — High Voltage Taser

1. **Introduction**
2. **Theoretical Background and Methodology**
    2.1 Joule Thief Circuit
    2.2 Cockcroft-Walton Voltage Multiplier
    2.3 Combining Joule Thief with Voltage Multiplier
3. **Circuit Diagram**
4. **Analysis**
5. **Hardware Implementation**
6. **Equipment and Cost**
7. **Project Outcome**
8. **Discussion**

SAP-1 Computer Architecture

Detailed Study Notes

---

1. Introduction to SAP Computers

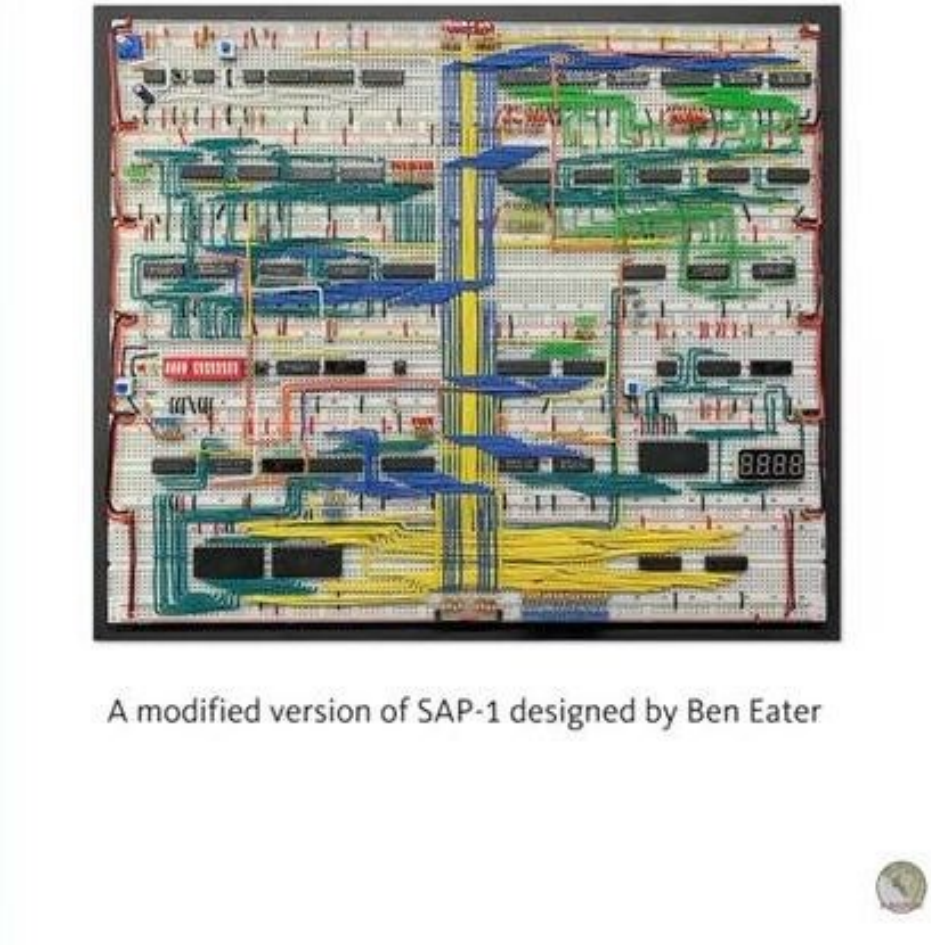


Fig 1 — SAP-1 introduction slide (a modified version built by Ben Eater)

SAP stands for **Simple-As-Possible**.

SAP computers (SAP-1, SAP-2, SAP-3) were first introduced by **Albert Paul Malvino** in his book **"Digital Computer Electronics"**.

- SAPs have a very simplified computer architecture.
- They help us understand the crucial ideas behind computer operation without unnecessary complexity.
- SAP-1 is the first evolution of SAPs.

2. Architecture of SAP-1

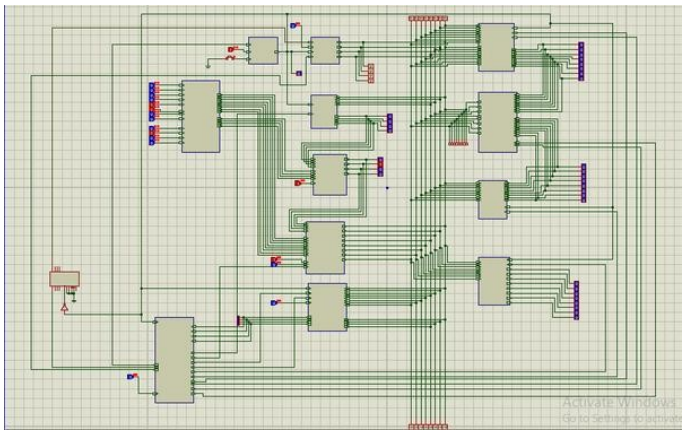


Fig — Full SAP-1 software simulation (circuit simulator)

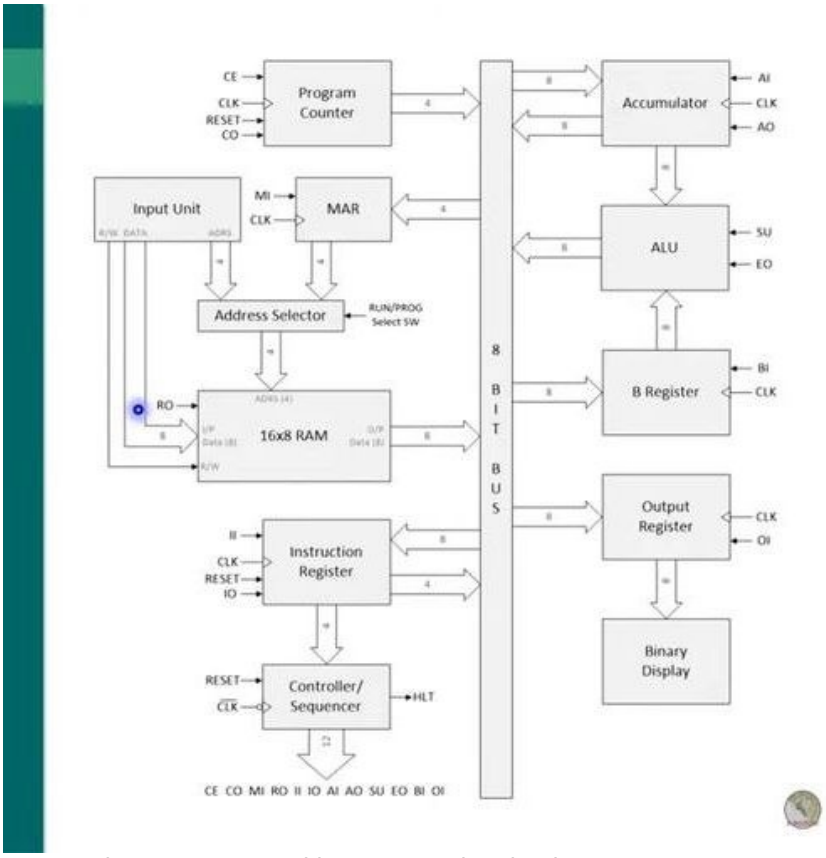
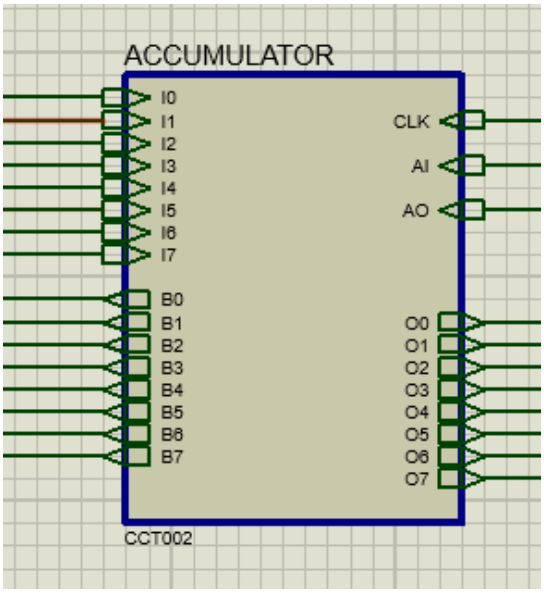


Fig 2a — SAP-1 architecture overview: hardware components

Hardware Components

- 8-bit bus
- 16×8 bit RAM
- A 4-bit Program Counter (PC)
- Input Unit and Memory Address Register (MAR)
- 8-bit Arithmetic and Logic Unit (ALU)
- Two 8-bit registers: Accumulator (Register A) and Register B



Software simulation — Accumulator circuit

- A controller/sequencer for controlling all other blocks
- Output unit for showing output in binary form

Instruction Set (5 Instructions)

SAP-1 executes exactly **5 instructions**. Each instruction is 8 bits: the upper 4 bits are the **opcode**, the lower 4 bits are the **address/operand**.

Mnemonic	Opcode (4-bit)	Full Name	Description
LDA	0000	Load A	Loads data from a specified RAM address into the Accumulator.
ADD	0001	Add	Adds data from a specified RAM address to the Accumulator. Result stored in Accumulator.
SUB	0010	Subtract	Subtracts data from a specified RAM address from the Accumulator. Result stored in Accumulator.
OUT	0011	Output	Loads data from the Accumulator and sends it to

			the Output Register for display.
HLT	1111	Halt	Halts the computer by stopping the main clock.

Clock Modes (2 Modes)

The main clock signal driving SAP-1 has two modes of operation:

Mode	Description
Auto	The clock oscillates automatically at a fixed frequency. The computer steps through instructions continuously without manual intervention.
Manual	Each clock pulse is triggered manually (e.g., by pressing a button). Useful for debugging and step-by-step observation of instruction execution.

3. The 8-bit Bus

The 8-bit bus consists of 8 wires/lines used for data transfer between all modules.

Bus Rules

Rule i: At any single time instance, only one module will send data to the bus, and only one module will receive that data.

Rule ii: Two modules can never send data to the bus at the same time — this causes **bus contention**.

Rule iii: When a module is sending data to the bus, all other modules' outputs must be disconnected. This is achieved using **tri-state buffers**.

Tri-State Buffer — 3 States

A tri-state buffer has **three possible output states**. It is the key component that allows modules to connect to and disconnect from the shared bus without causing contention.

State Name	Enable (En)	Output	Meaning
Logic HIGH	**1**	Out = 1 (HIGH)	En=1 allows input through. Module actively drives bus HIGH.
Logic LOW	**1**	Out = 0 (LOW)	En=1 allows input through. Module actively drives bus LOW.
High-Z (Disconnected)	**0**	Floating (Hi-Z)	En=0 disconnects output entirely — open circuit. Module is fully isolated from the bus. This is the key state that prevents bus contention.

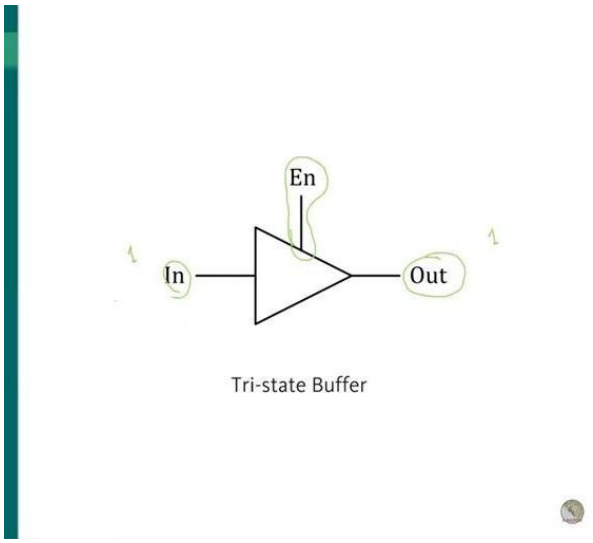
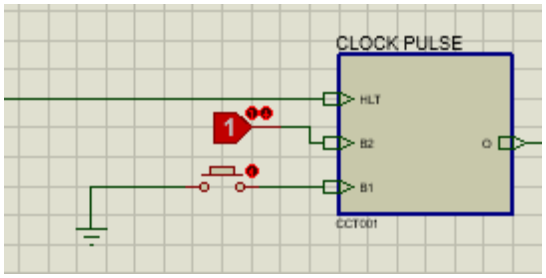


Fig 3a — Tri-state buffer symbol

4. Program Counter (PC)



Software simulation — Clock Pulse circuit

Fig 4 — Program Counter module and its connections

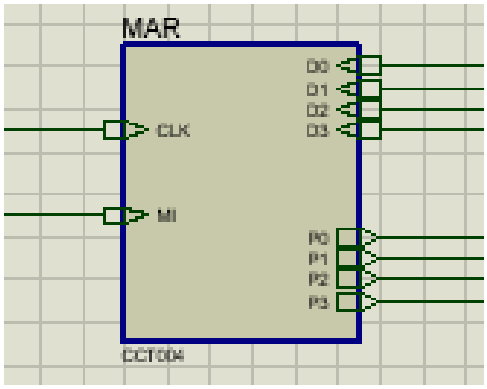
The Program Counter is a **4-bit counter** counting from **0000** to **1111**.

- Stores and sends out the memory address of the next instruction to be fetched.
- Increments its value after each instruction cycle.

Control Bits

Control Signal	Abbr.	Description
Counter Enable	**CE**	Increments the program counter on the next clock cycle.
Counter Out	**CO**	Puts the current program counter value on the bus. Output is disconnected when CO is low.

5. Memory Address Register (MAR)



Software simulation — MAR circuit

Fig 5 — MAR module and its connections to Address Selector and RAM

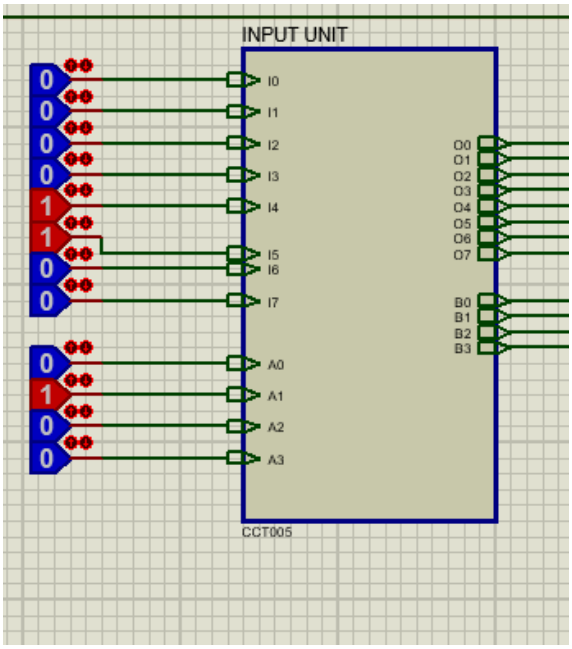
The MAR is a 4-bit register.

- Stores the memory address of RAM from which data will be fetched.
- This 4-bit address is obtained via the bus from the Program Counter and then stored.
- The stored address is sent to the RAM where data or instructions are read from.

Control Bits

Control Signal	Abbr.	Description
Memory Address In	**MI**	Stores the current values of the bus into the Memory Address Register (MAR).

6. Input Unit



Software simulation — Input Unit circuit

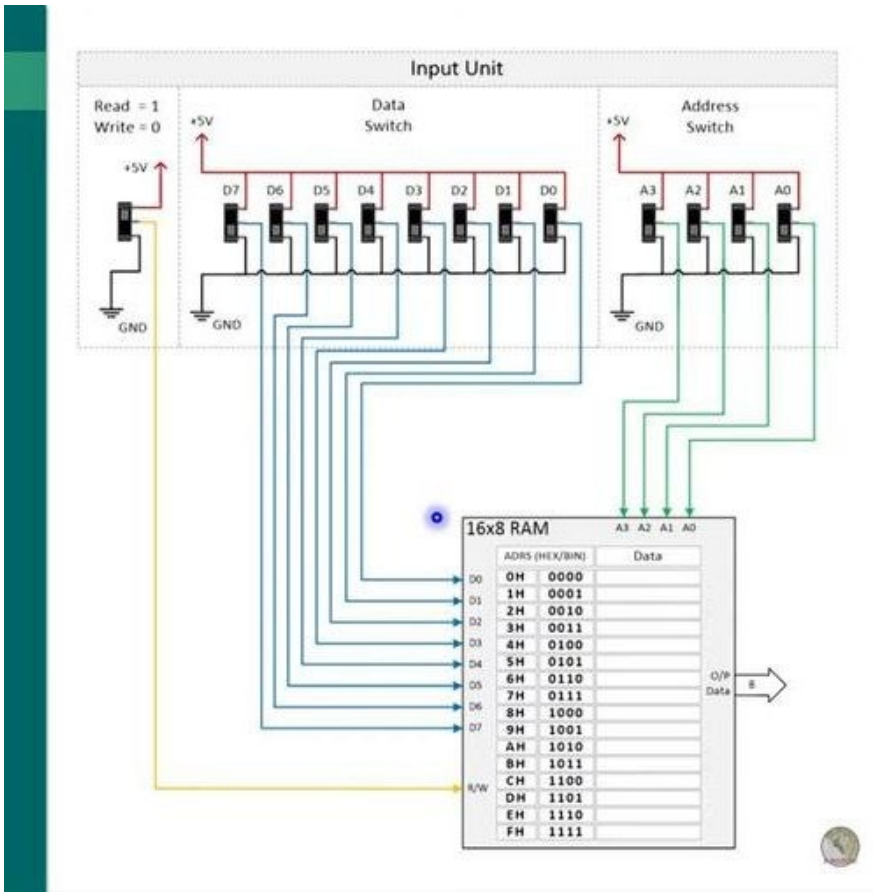


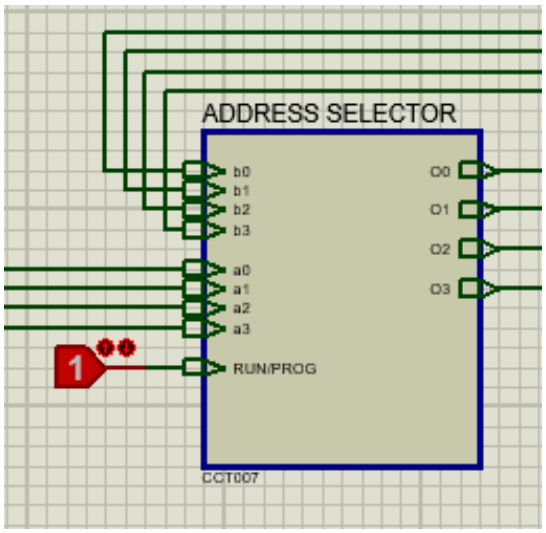
Fig 6 — Input Unit switch layout and connection to RAM

The Input Unit is used when the computer needs to be programmed by writing data into the RAM — the **PROG state**.

Switches

Switch	Description
DATA Switch	8 switches (D7–D0) for writing 8-bit data to RAM.
ADRS Switch	4 switches (A3–A0) for accessing a particular RAM address for writing.
R/W Switch	Switches between read (1) and write (0) mode.

7. RUN/PROG Selector Switch



Software simulation — Address Selector circuit

The computer has two operating states:

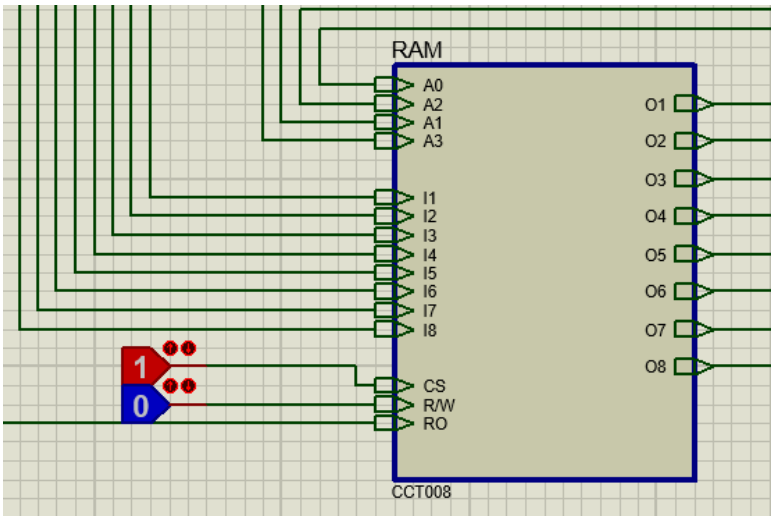
RUN: When a program is in execution on the computer.

PROG: Before running, the computer must be programmed by writing data into the RAM.

Based on the RUN/PROG state, a 4-bit address is selected from either **MAR** or **Input Unit** using the **Address Selector**.

State	Address Source Selected
RUN	MAR will be selected
PROG	Input Unit will be selected

8. Random Access Memory (RAM)



Software simulation — 16×8 RAM circuit

RAM size: **16×8 bit** — 16 memory locations, each storing 8 bits.

- The RAM is programmed using the address switches and data switches of the input unit.
- During a computer run, the RAM receives its 4-bit address from the MAR.

Control Bits

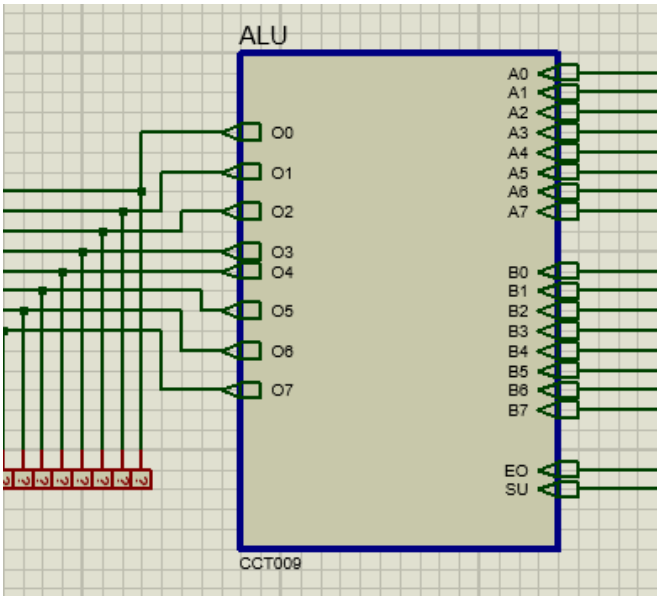
Control Signal	Abbr.	Description
RAM Out	**RO**	Puts the currently selected RAM byte onto the bus. Output is disconnected when RO is low.

Writing to RAM — Example

Write 8-bit data **0110 1100** at address **1001**:

- **D7–D0 DATA switches:** set to 0110 1100
- **A3–A0 ADRS switches:** set to 1001
- **R/W switch:** set to Write (0)
- Toggle write cycle to store data at address 9H.

9. Arithmetic and Logic Unit (ALU)



Software simulation — ALU circuit

The ALU performs 8-bit binary addition and subtraction.

Control Bits

Control Signal	Abbr.	Description
Addition/Subtraction Select	**SU**	SU=0: addition operation | SU=1: subtraction operation
ALU Out	**EO**	Puts the ALU output onto the bus. Output is disconnected when EO is low.

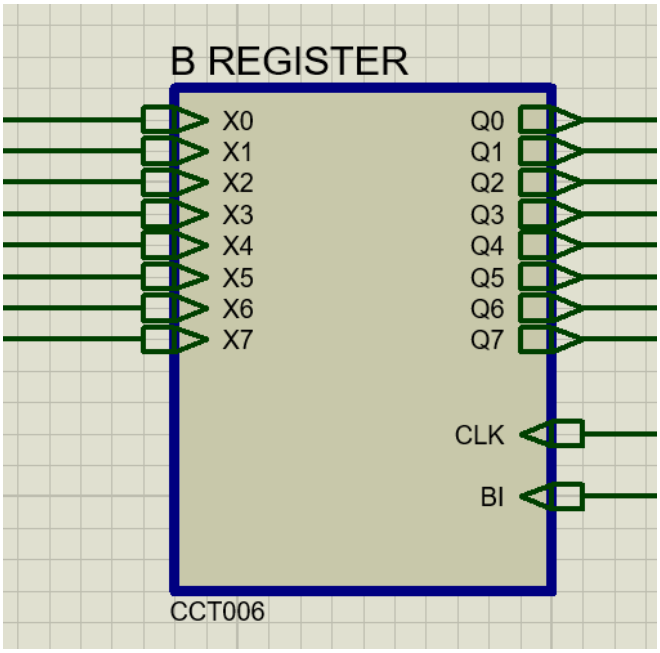
Accumulator (Register A)

The Accumulator is an 8-bit register that:

- Stores one operand for ALU operations.
- Receives and holds the result after ALU computation.
- Its output is always connected internally to the ALU (permanent connection, independent of bus).

Control Signal	Abbr.	Description
Accumulator In	**AI**	Loads the current value on the bus into the Accumulator on the next clock edge.
Accumulator Out	**AO**	Puts the Accumulator value onto the 8-bit bus. Output is disconnected when AO is low (tri-state buffer).

B Register



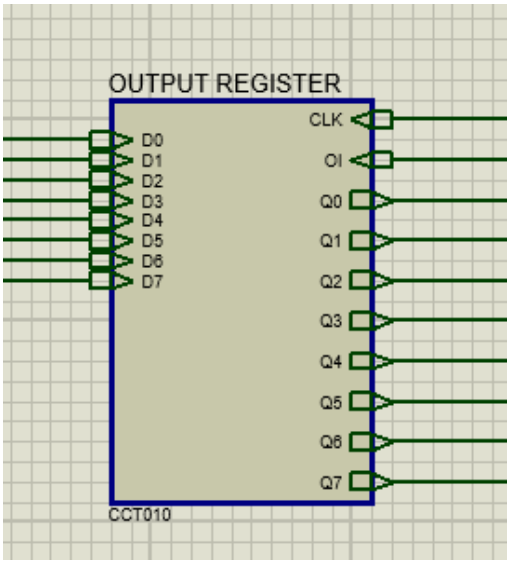
Software simulation — B Register circuit

The B Register is an 8-bit register that:

- Stores the second operand for ALU operations.
- Its output is always fed directly into the ALU (permanent internal connection, not via bus).
- Unlike the Accumulator, the B Register has no output to the bus — it only receives data.

Control Signal	Abbr.	Description
B Register In	**BI**	Loads the current value on the bus into the B Register. This is then immediately available to the ALU as the second operand.

10. Output Unit



Software simulation — Output Register circuit

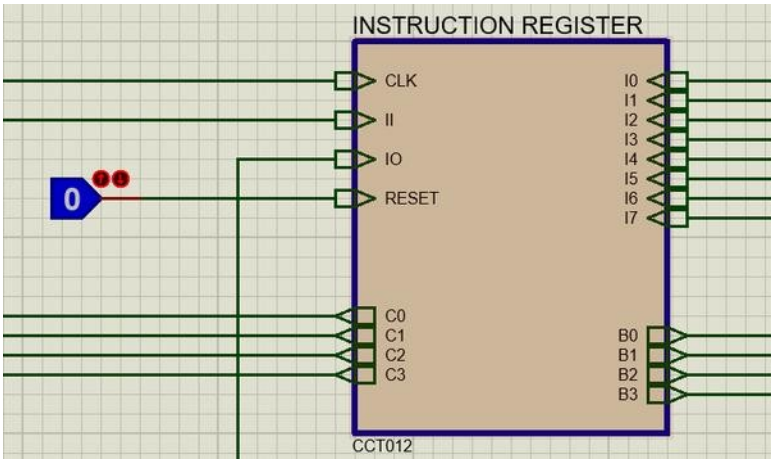
The Output Unit is an 8-bit register that:

- Stores the result to display when requested.
- Receives data from the Accumulator via the bus when the OUT instruction is executed.
- Connects to a Binary Display (LEDs) for showing output

Control Bits

Control Signal	Abbr.	Description
Output Register In	**OI**	Stores the current values of the bus into the Output Register.

11.1 Instruction: SUB (Subtract)



Software simulation — Instruction Register circuit

Instruction	Subtract
Mnemonic	**SUB**
Opcode	**0010**

Function: Subtracts the value stored at a particular RAM address from the value currently in the Accumulator. The RAM address is given by the address field (lower 4 bits) of the instruction.

Instruction format (8 bits): **0010 AAAA** — where AAAA is the 4-bit RAM address

Example	
Assembly	**SUB BH**
In Binary	**0010 1011**

Explanation: Opcode **0010** = SUB. Address **1011** = BH (address 11 in hex). The computer subtracts the byte stored at RAM address BH from the Accumulator and stores the result back in the Accumulator.

11.2 Instruction: OUT (Output)

Instruction	Output
Mnemonic	**OUT**
Opcode	**0011**

Function: Loads data from the Accumulator and sends it to the Output Register. The Output Register then drives the Binary Display.

Instruction format (8 bits): **0011** 0000 — address field is unused (set to 0000)

Example	
Assembly	**OUT**
In Binary	**0011 0000**

Explanation: Opcode **0011** = OUT. No address field is needed. The current value of the Accumulator is transferred to the Output Register and shown on the binary display.

11.3 Instruction: HLT (Halt)

Fig 11.3 — HLT instruction slide

Instruction	Halt
Mnemonic	**HLT**
Opcode	**1111**

Function: Halts the process of the computer by stopping the main clock. All execution ceases immediately.

Instruction format (8 bits): **1111** 0000 — address field is unused

Example	
Assembly	**HLT**
In Binary	**1111 0000**

Explanation: Opcode **1111** = HLT. The Controller/Sequencer detects this opcode and stops the clock, freezing the computer in its current state.

12. Complete Control Signals Summary

Module	Signal	Full Name	Function
Program Counter	**CE**	Counter Enable	Increment PC on next clock cycle
Program Counter	**CO**	Counter Out	Put PC value on bus
MAR	**MI**	Memory Address In	Load bus value into MAR
RAM	**RO**	RAM Out	Put RAM byte on bus
Instruction Reg.	**II**	Instruction In	Load bus value into IR
Instruction Reg.	**IO**	Instruction Out	Put lower 4 bits of IR on bus
Accumulator	**AI**	Accumulator In	Load bus value into Accumulator
Accumulator	**AO**	Accumulator Out	Put Accumulator value on bus

ALU	**SU**	Subtract Select	0 = Add, 1 = Subtract
ALU	**EO**	Enable Out	Put ALU result on bus
B Register	**BI**	B Register In	Load bus value into B Register
Output Register	**OI**	Output In	Load bus value into Output Register

13. Instruction Cycle and T States

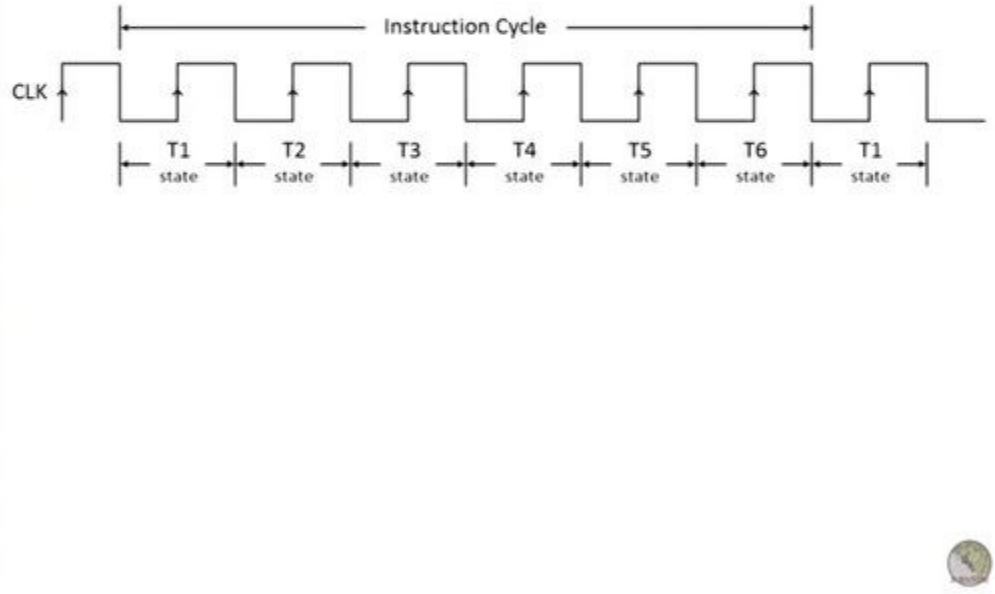


Fig 13 — One instruction cycle = 6 T states (T1–T6), each one clock pulse wide

What is an Instruction Cycle?

The **instruction cycle** is the total time required to fetch and execute one single instruction. In SAP-1, every instruction cycle consists of exactly **6 T states**.

What is a T State?

A **T state** (time state) is the time required to complete one **microinstruction** — a single atomic operation (e.g. put PC on bus, load MAR). Each T state lasts exactly **one clock pulse**.

Structure of an Instruction Cycle

An instruction cycle is divided into two sub-cycles:

Sub-cycle	T States	Purpose
Fetch Cycle	**T1, T2, T3**	Fetches the next instruction from RAM into the Instruction Register. Identical for every instruction.
Execution Cycle	**T4, T5, T6**	Executes the fetched instruction. Varies depending on which instruction is being executed.

Key insight: Because the fetch cycle is always the same, T1–T3 microinstructions are identical for every instruction in SAP-1. Only T4–T6 differ between instructions.

14. Fetch and Execution Cycles

14.1 Fetch Cycle (T1, T2, T3)

The fetch cycle is **identical for all instructions**. It retrieves the instruction stored at the address currently held in the Program Counter and loads it into the Instruction Register.

T State	State Name	Activated Control Bits
T1	Address State	**CO MI**
T2	Increment State	**CE**
T3	Memory State	**RO II**

Step-by-Step: What Happens in Each Fetch State

T1 — Address State (CO, MI): The Program Counter outputs its current value onto the bus (CO active). The Memory Address Register loads this value from the bus (MI active). Now the MAR holds the address of the next instruction.

T2 — Increment State (CE): The Program Counter increments by 1 (CE active), so it is ready to point to the next instruction for the following cycle. The bus is not used in this step.

T3 — Memory State (RO, II): The RAM outputs the byte stored at the address in MAR onto the bus (RO active). The Instruction Register loads this byte from the bus (II active). The instruction is now inside the IR, ready to be decoded and executed.

14.2 Execution Cycle (T4, T5, T6)

During the execution cycle the **Controller/Sequencer** reads the opcode from the Instruction Register and activates the appropriate control bits for T4, T5, and T6. These **vary per instruction**.

Execution Cycle — LDA (Load Accumulator)

T State	Activated Control Bits
T4	**IO MI**
T5	**AI RO**
T6	No operation

T4 (IO, MI): The lower 4 bits of the Instruction Register (the address field) are put on the bus (IO active). The MAR loads this address (MI active).

T5 (AI, RO): The RAM outputs the byte at the address now in MAR (RO active). The Accumulator loads this byte (AI active). The load is complete.

T6: No operation — LDA only needs 2 execution steps.

Execution Cycle — ADD

ADD:

T State	Activated Control Bits
T4	**IO MI**
T5	**RO BI**
T6	**EO AI**

T4 (IO, MI): Address field of instruction → bus → MAR.

T5 (RO, BI): RAM byte at that address → bus → B Register (second ALU operand).

T6 (EO, AI): ALU computes Accumulator + B Register (SU=0 so addition). Result → bus → Accumulator.

Execution Cycle — SUB

T State	Activated Control Bits
T4	**IO MI**
T5	**RO BI**

T6	**EO AI SU**

Identical to ADD for T4 and T5. At T6, **SU is also activated** alongside EO and AI, which tells the ALU to subtract instead of add (Accumulator − B Register). Result stored back in Accumulator.

Execution Cycle — OUT

T State	Activated Control Bits
T4	**AO OI**
T5	No operation
T6	No operation

T4 (AO, OI): Accumulator value → bus → Output Register. The Binary Display immediately reflects the new value. Only one step needed.

Execution Cycle — HLT

HLT does not require any execution cycle states. No registers are involved. The Controller/Sequencer simply activates the **HLT bit**, which stops the main clock immediately. The computer freezes in its current state.

14.3 Complete T-State Summary — All Instructions

Instr.	T1	T2	T3	T4	T5
LDA	CO MI	CE	RO II	**IO MI**	**AI RO / —**
ADD	CO MI	CE	RO II	**IO MI**	**RO BI / EO AI**
SUB	CO MI	CE	RO II	**IO MI**	**RO BI / EO AI SU**
OUT	CO MI	CE	RO II	**AO OI**	**— / —**
HLT	CO MI	CE	RO II	**HLT**	**— / —**

Note: T1–T3 (grey) are the **fetch cycle** — same for all instructions. T4–T6 (blue) are the **execution cycle** — unique per instruction.

15. Example Program

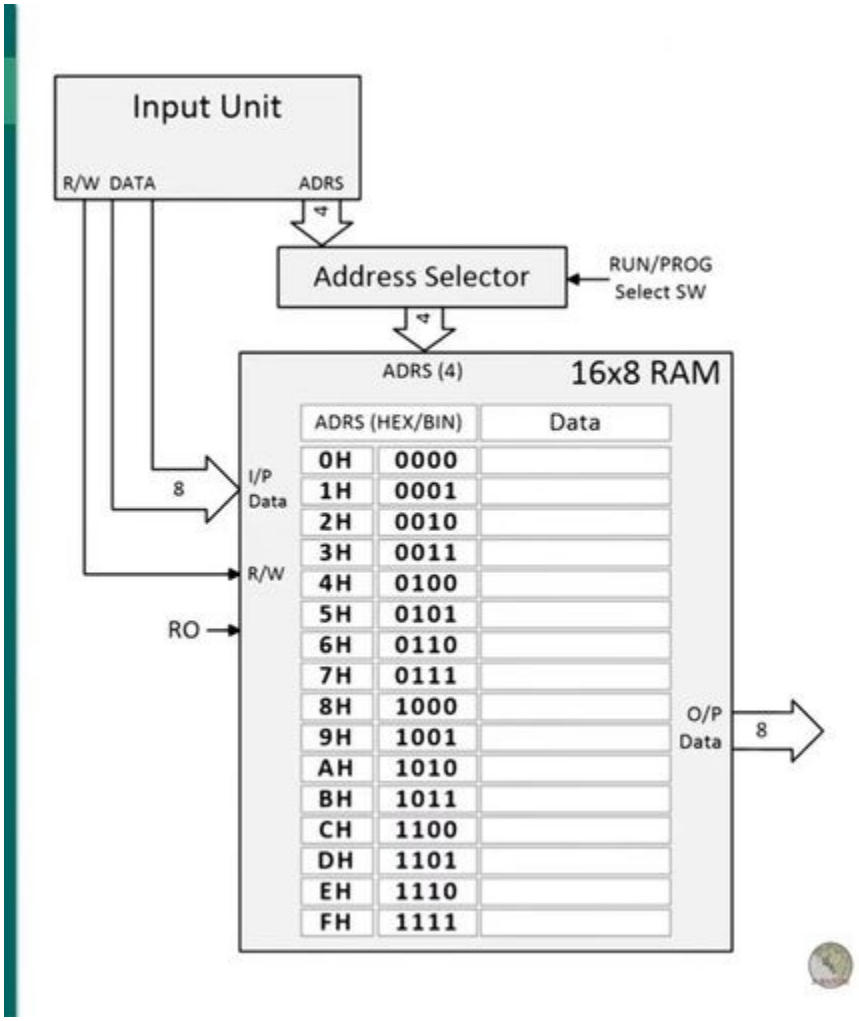


Fig 15a — Example program loaded into RAM (mnemonic form)

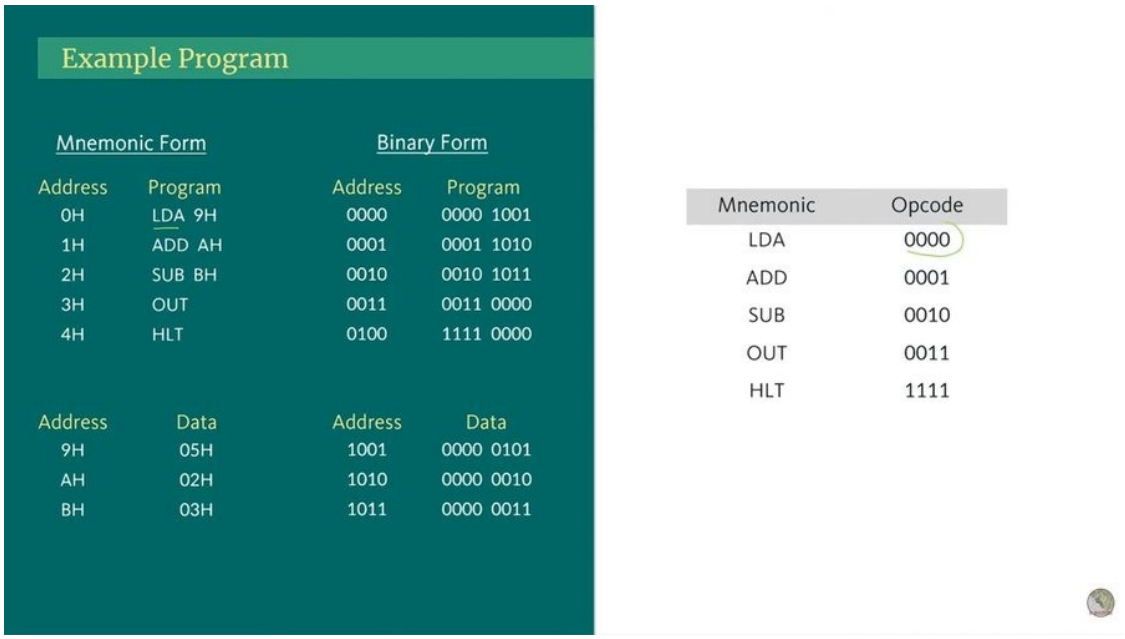


Fig 15b — Mnemonic form, binary form, and opcode reference

Program Description

This example program loads two values from memory, adds and subtracts them, outputs the result, then halts.

Program in Mnemonic Form

Address	Instruction
0H (0000)	LDA 9H — Load value at address 9H into Accumulator
1H (0001)	ADD AH — Add value at address AH to Accumulator
2H (0010)	SUB BH — Subtract value at address BH from Accumulator
3H (0011)	OUT — Output Accumulator value to display
4H (0100)	HLT — Halt the computer

Data Stored in Higher Memory

Address (Hex)	Address (Binary)	Data (Hex)	Data (Binary)
9H	**1001**	**05H**	0000 0101
AH	**1010**	**02H**	0000 0010
BH	**1011**	**03H**	0000 0011

Program in Binary Form

Address (Binary)	Instruction (Binary)	Breakdown
0000	**0000 1001**	Opcode 0000 (LDA) + Address 1001 (9H)
0001	**0001 1010**	Opcode 0001 (ADD) + Address 1010 (AH)
0010	**0010 1011**	Opcode 0010 (SUB) + Address 1011 (BH)
0011	**0011 0000**	Opcode 0011 (OUT) + Unused 0000
0100	**1111 0000**	Opcode 1111 (HLT) + Unused 0000

Step-by-Step Execution Trace

What the computer actually computes when this program runs:

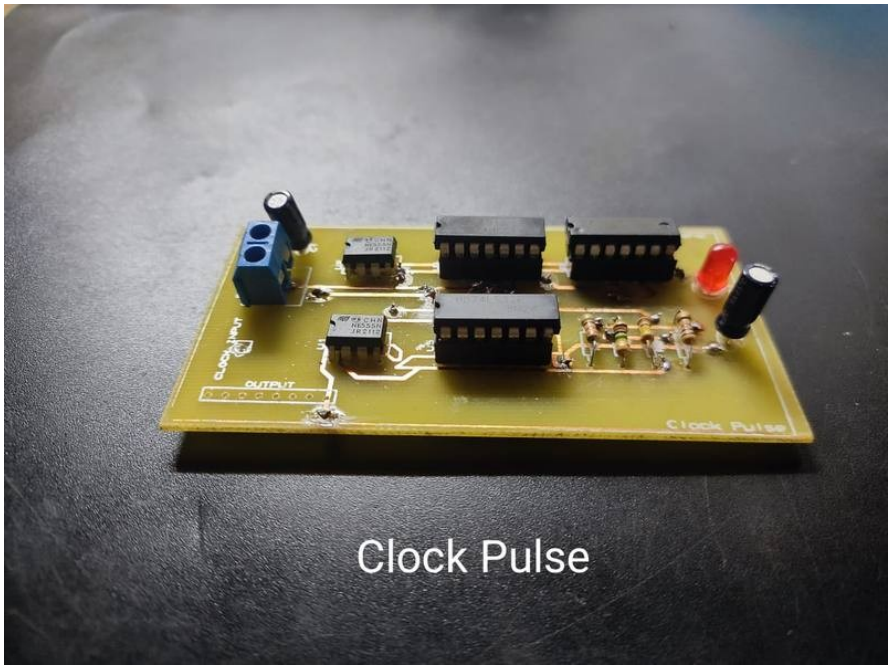
- **LDA 9H:** Accumulator ← 05H (load 5)
- **ADD AH:** Accumulator ← 05H + 02H = 07H (add 2, result = 7)
- **SUB BH:** Accumulator ← 07H − 03H = 04H (subtract 3, result = 4)
- **OUT:** Output Register ← 04H (display: 0000 0100 in binary)
- **HLT:** Clock stops. Final displayed value = 4 (decimal)

16. Hardware Implementation (PCB)

The following photographs show the physical PCB implementations of each SAP-1 module, designed and assembled by Team SAPiens at the Islamic University of Technology (IUT). Each module was built as a separate PCB and interconnected via the shared 8-bit bus.

Manufactured by Taru PCB. Estimated cost: PCB ~4,000 BDT | Components ~9,500 BDT | Miscellaneous ~3,000 BDT | Total ~16,500 BDT

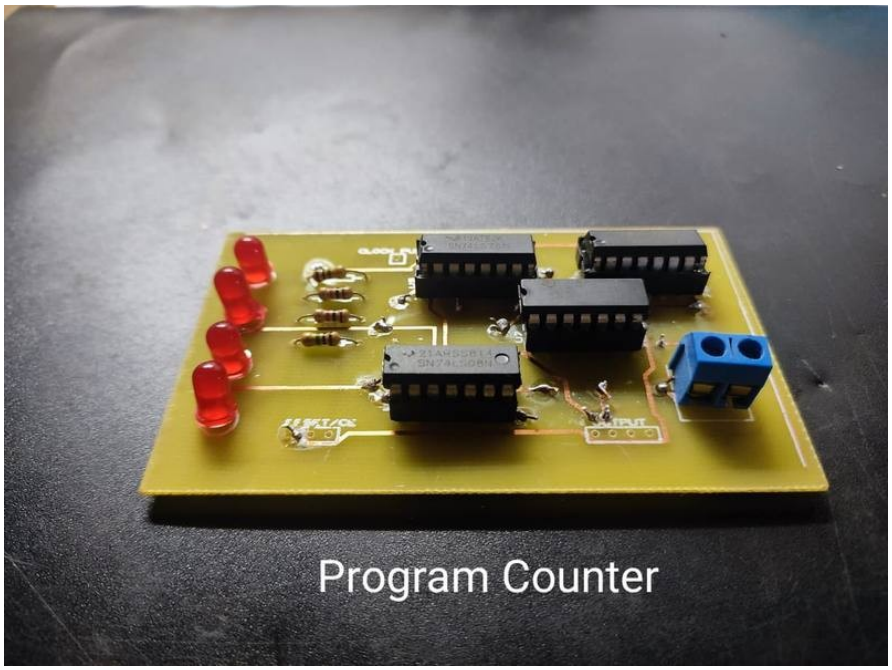
16.1 Clock Pulse



PCB — Clock Pulse

555 Timer IC in Astable mode (auto) and Monostable mode (manual push-button). Drives the master clock line for all modules.

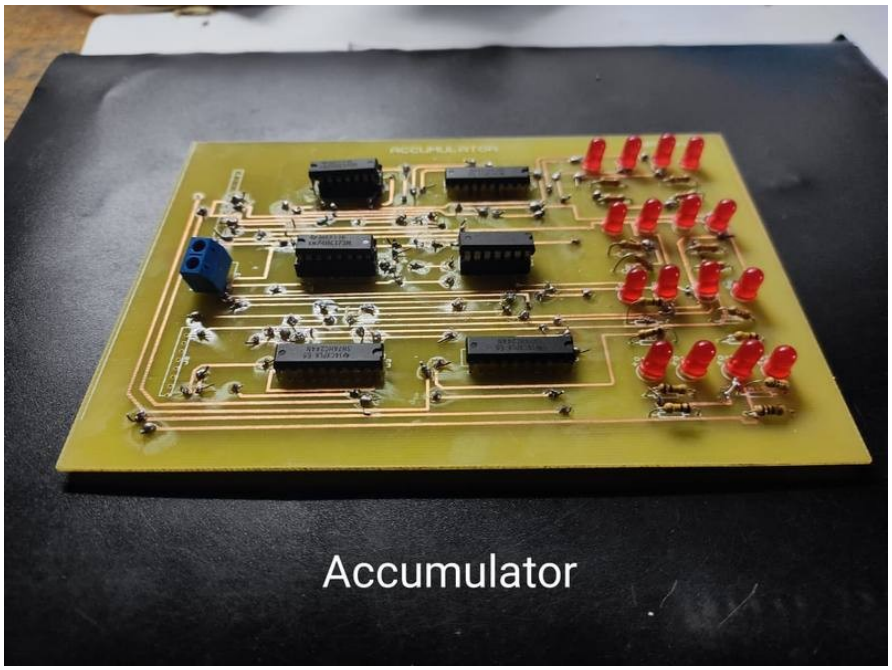
16.2 Program Counter



PCB — Program Counter

4-bit synchronous up-counter (0000→1111) built with 4 JK Flip-Flops. CE increments on each clock cycle; CO drives the address onto the bus.

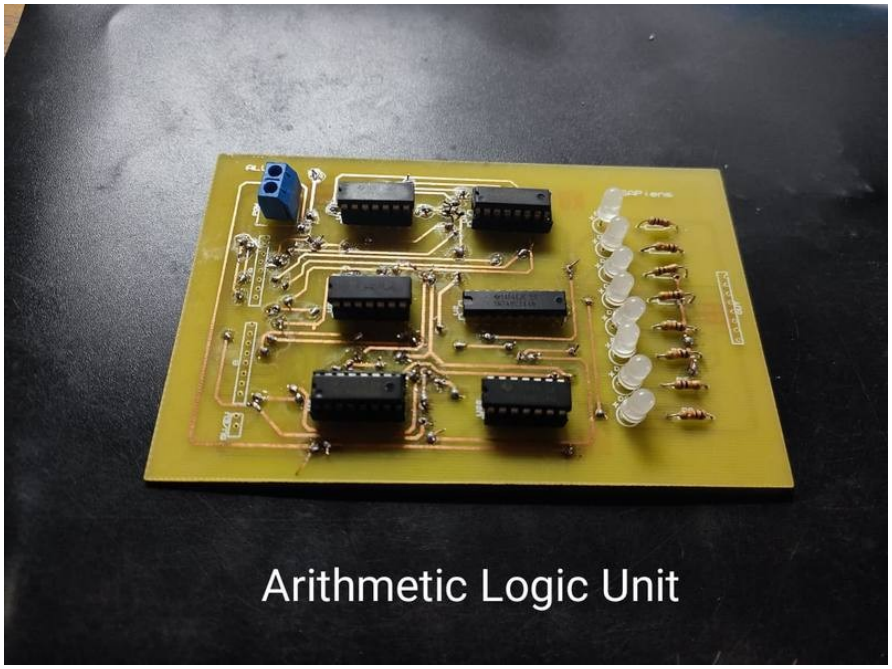
16.3 Accumulator / Register A



PCB — Accumulator / Register A

8-bit register holding one ALU operand and storing the result. Permanent internal connection to the ALU. AI loads from bus; AO drives bus via tri-state buffer.

16.4 Arithmetic Logic Unit (ALU)



PCB — Arithmetic Logic Unit (ALU)

8-bit addition (SU=0) and subtraction (SU=1) using 2's complement. EO enables the ALU result onto the bus; disconnected (Hi-Z) when EO is low.

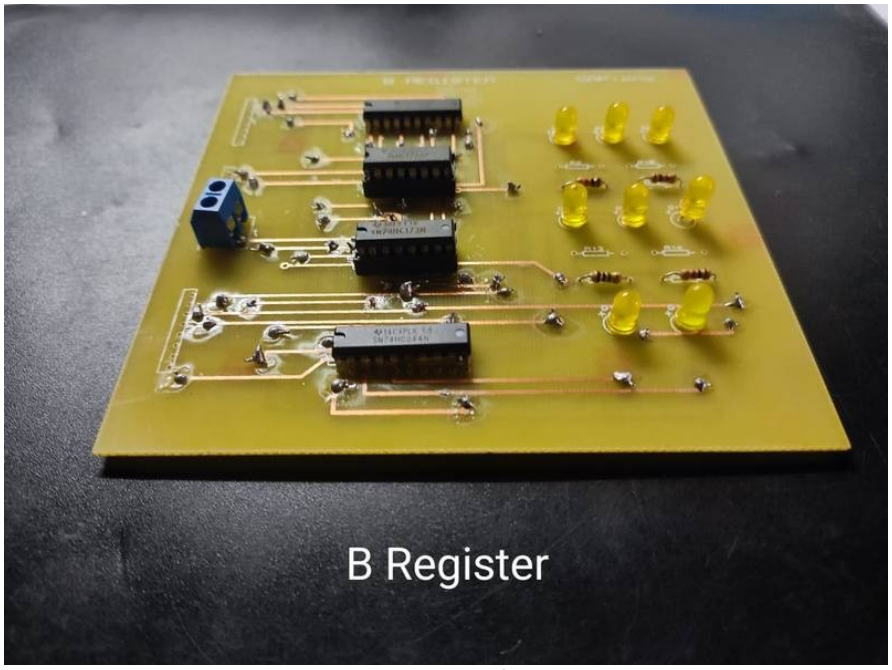
16.5 RAM / MAR / Input Unit / Address Selector



PCB — RAM / MAR / Input Unit / Address Selector

Combined PCB: 16×8-bit RAM, 4-bit MAR (74HC173), 8 DATA + 4 ADRS input switches, and Address Selector (logic gates). RUN mode → MAR addresses RAM; PROG mode → switches address RAM.

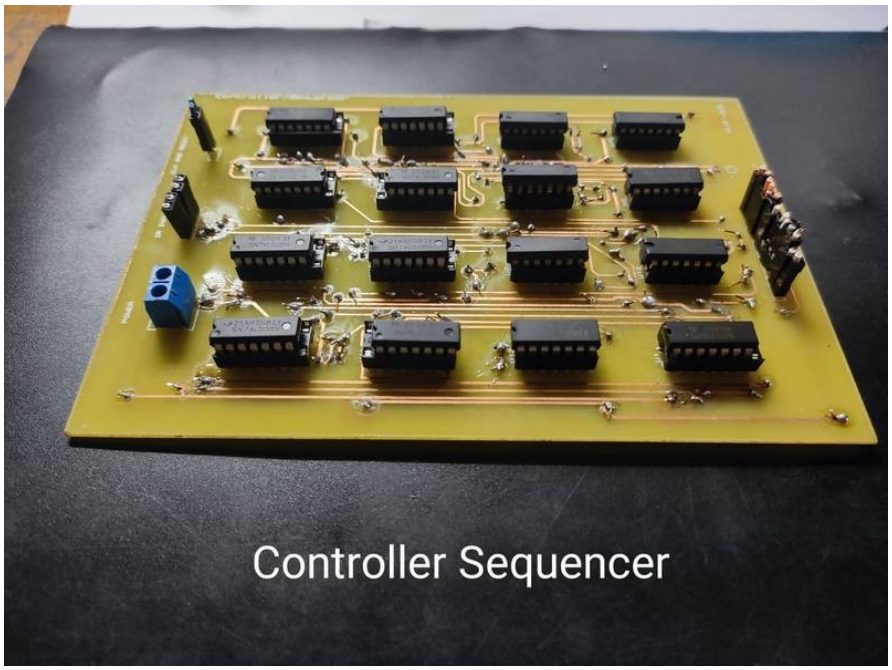
16.6 B Register



PCB — B Register

8-bit register for the ALU's second operand via permanent internal connection. BI loads from bus; no bus output (write-only from bus perspective).

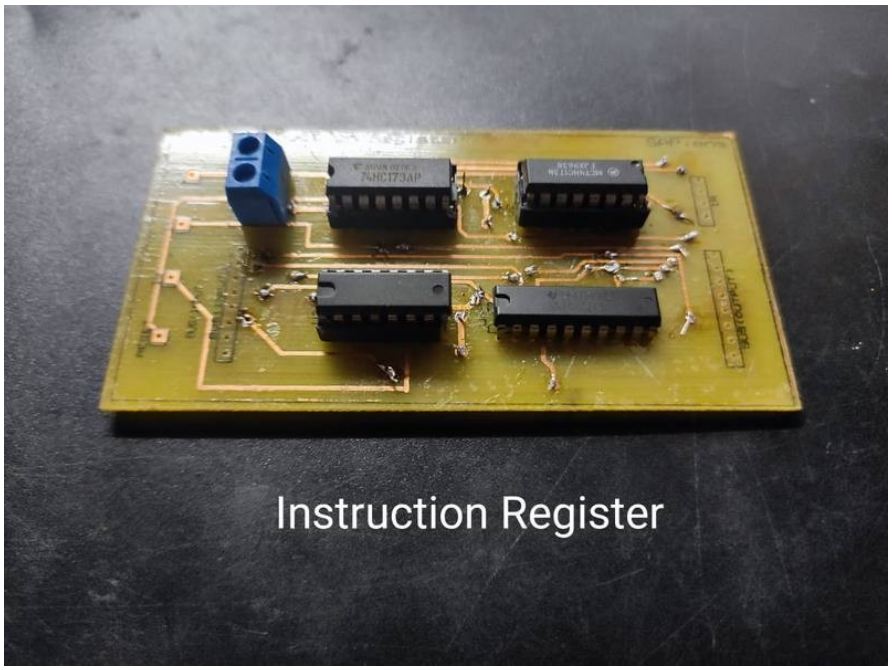
16.7 Controller / Sequencer



PCB — Controller / Sequencer

Generates all 12 control bits (CE CO MI RO II IO AI AO SU EO BI OI) + HLT each T state. Implemented with 5 T states (T2 and T3 merged) due to PCB manufacturing issues and time constraints.

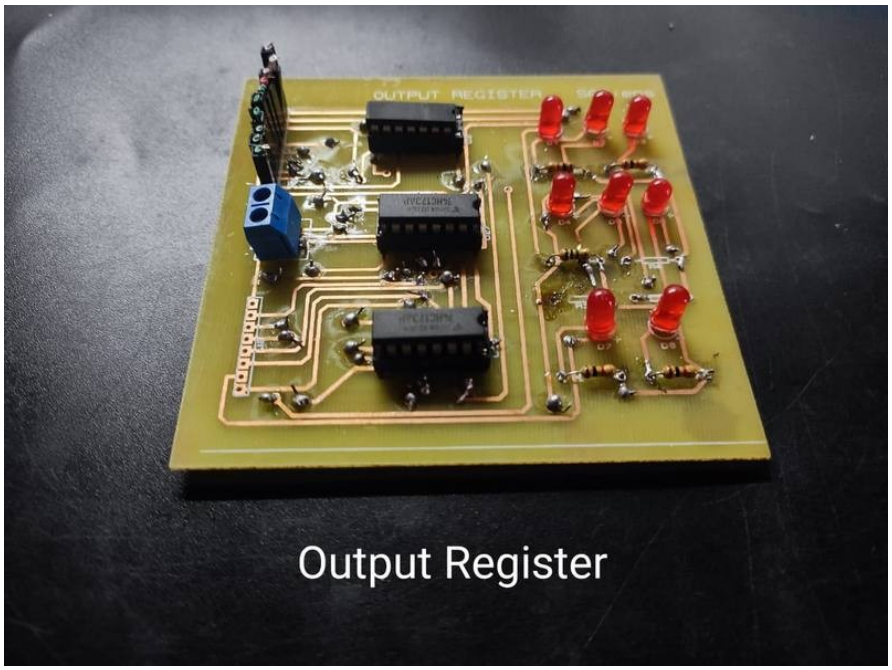
16.8 Instruction Register



PCB — Instruction Register

Stores the 8-bit fetched instruction. Upper nibble (opcode) feeds permanently to the Controller/Sequencer; lower nibble (address field) is driven onto the bus when IO is active.

16.9 Output Register



PCB — Output Register

8-bit register capturing the Accumulator's value on OUT instruction (OI active). Drives the binary LED display. 7-segment display was not implemented due to time constraints.

Project Nema
An Automated Aquarium

Project Report

Abstract

This project report presents the development of an innovative automated aquarium system leveraging Arduino technology. The primary objective is to create a comprehensive ecosystem within the aquarium through the integration of various sensors and actuators. The system incorporates pH sensors, a water level indicator, temperature control mechanisms, a GSM module for remote monitoring, and a servo motor for precise and automated feeding. The implementation allows for real-time monitoring and control, ensuring optimal environmental conditions for aquatic life while enabling remote accessibility for seamless management.

Key Points

pH Sensors	**Water Level Indicator**	**Temperature Control**
GSM Module	**Servo Motor / Auto Feeding**	**Arduino Automation**
Real-Time Monitoring	**Remote Accessibility**	**IoT Implementation**

1. Introduction

1.1 Purpose

In the realm of aquatic habitat maintenance, the amalgamation of technology and ecological preservation has taken a pioneering leap. This project endeavors to redefine the paradigm of aquarium management, where precision meets environmental sensitivity through the fusion of cutting-edge technology and intricate ecological understanding.

1.2 Objectives

- **Ecosystem Harmony:** Forge an automated system that actively sustains a harmonious aquatic ecosystem by regulating critical environmental parameters such as pH levels and temperature.
- **Technical Ingenuity:** Confront the complexities of integrating diverse technological components — GSM modules, servo motors, and precise water-level control mechanisms.
- **Environmental Precision:** Attain a level of precision that caters to the nuanced needs of aquatic life, ensuring a stable and conducive environment for their well-being and longevity.

- **Symbiosis of Automation and Ecology:** Bridge the gap between technological automation and ecological sensitivity, demonstrating how sophisticated systems can coexist with the natural environment within an aquarium.

2. Overview of Components

2.1 Principal Components

2.1.1 Arduino Uno



Fig 2.1 — Arduino Uno (central microcontroller)

The Arduino Uno acts as the central intelligence in the automated aquarium project, orchestrating sensor data interpretation and precise control of actuators. It manages pH sensors, water level indicators, temperature controls, a GSM module for remote monitoring, and servo motors for automated feeding. Its scalability allows seamless integration of new sensors or features.

2.1.2 GSM Module (SIM800L)



Fig 2.2 — SIM800L GSM Module

Integrated within the aquarium setup, the GSM module enables instant communication with the user. Coupled with the Arduino Uno, this module transmits real-time updates and notifications directly to the user's mobile device. In case of any deviations in vital parameters (pH or temperature beyond set thresholds), the GSM module triggers immediate SMS alerts.

2.1.3 Temperature Sensor

The temperature sensor serves as the vigilant guardian of thermal stability. Linked to the Arduino Uno, it continuously monitors water temperature with precision, ensuring the aquatic environment remains within the predefined range vital for the well-being of aquatic life. When temperature drops below the threshold, it actively triggers heater activation.

2.1.4 pH Sensor

At the core of the system's chemical equilibrium lies the pH sensor, functioning as the custodian of water quality. It vigilantly scrutinizes pH levels — a critical factor in sustaining a healthy aquatic environment. In the event of pH fluctuations beyond optimal ranges (pH > 10), the sensor triggers the GSM module to send an immediate alert.

2.1.5 Water Level Indicator (SONAR / HC-SR04)



Fig 2.3 — HC-SR04 Ultrasonic Sensor (SONAR)

The SONAR-based water level indicator uses ultrasonic waves to precisely measure water levels. Integrated with the Arduino Uno, it ensures the aquarium maintains an ideal water volume, triggering necessary pump actions to maintain stability.

2.1.6 Heater

Paired with the Arduino Uno, the heater uses precise temperature control for maintaining optimal water conditions. When temperature falls below 25°C, the Arduino activates the heater via the digital output pin, with an LED indicator illuminating simultaneously to signal activation.

2.1.7 Water Pump



Fig 2.4 — Water Pump

Integrated with the Arduino Uno, the water pump facilitates periodic water changes, enhancing circulation and maintaining water quality. Both inlet and outlet pumps are controlled by Arduino digital pins, and can be triggered remotely via a GSM command ('m').

2.1.8 Servo Motor

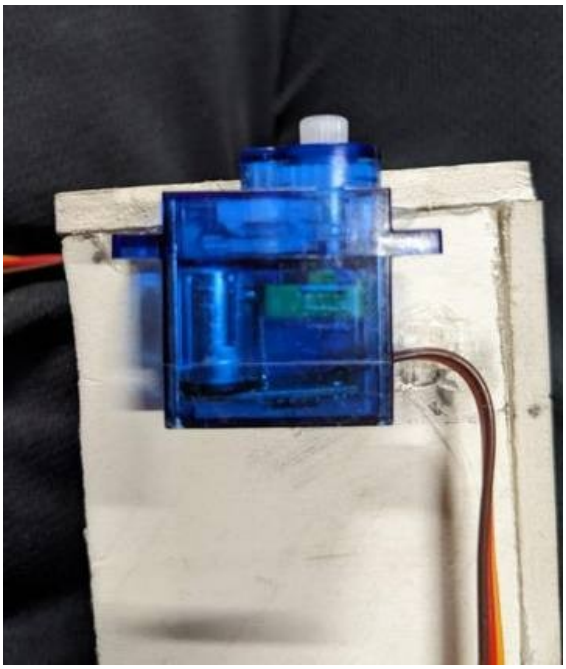


Fig 2.5 — Servo Motor (fish feeder mechanism)

Integrated with the Arduino Uno, the servo motor enables precise and automated fish feeding by controlling the dispensing mechanism. After a 12-hour interval, the Arduino triggers the servo motor to rotate to 45° (dispensing food) then returns to 0°, ensuring consistent scheduled nourishment.

2.2 Basic Components

Component	Component	Component
Vero Board	Bread Board	Resistor
MOSFET	OLED Display	Jumper Wire
LED Bulb	Pipes	Demo Aquarium

3. System Operation Overview

3.1 Methodology Flow

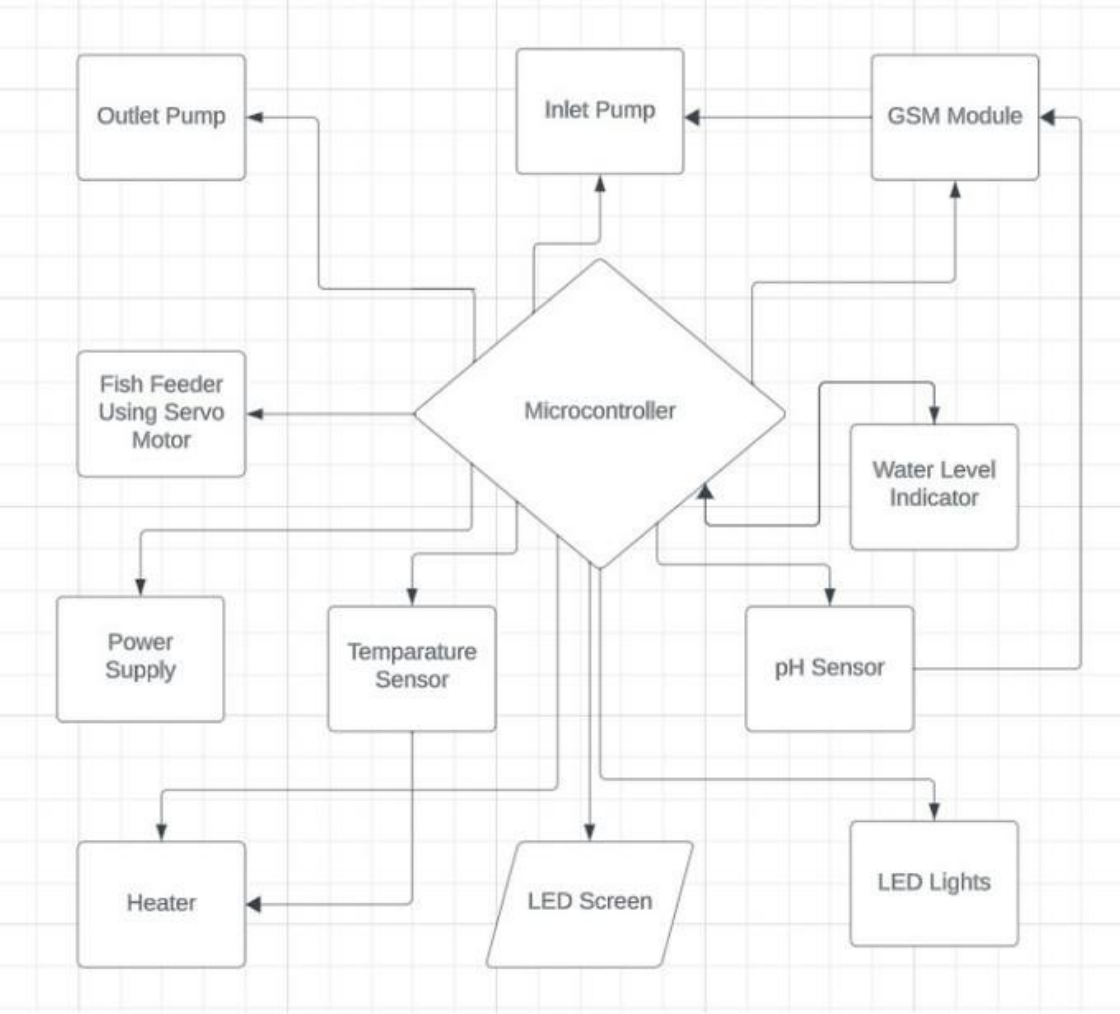


Fig 3.1 — System methodology flow diagram

The microcontroller (Arduino Uno) sits at the center of all operations, interfacing bidirectionally with every module. Sensors feed data in; the controller decides what actuators to trigger; users interact remotely via the GSM module.

3.2 Circuit Diagrams

3.2.1 GSM Module — Arduino Connection

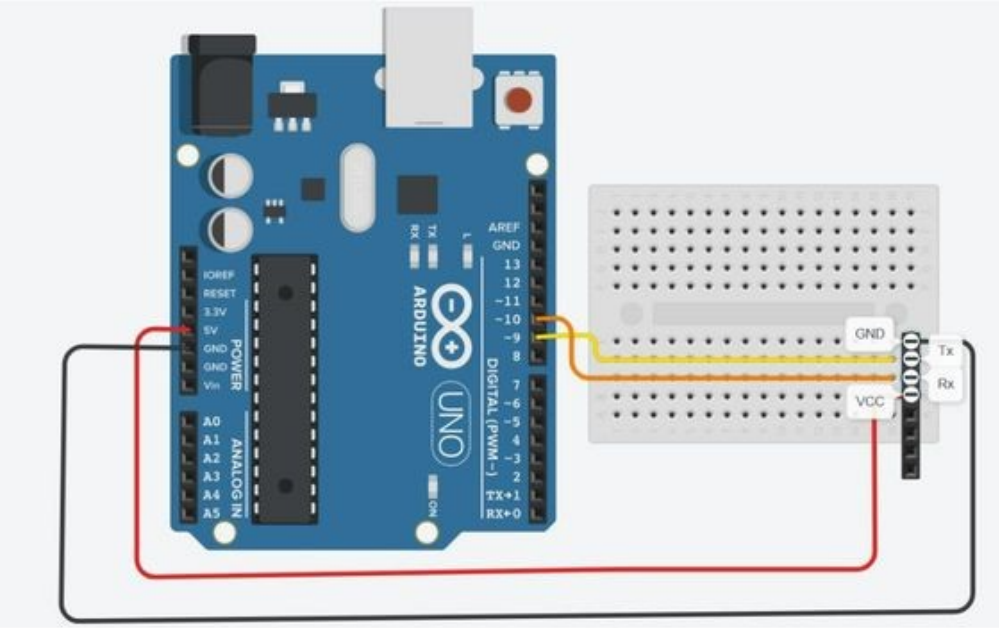


Fig 3.2 — GSM module wiring to Arduino Uno

3.2.2 Temperature Sensor — Arduino Connection

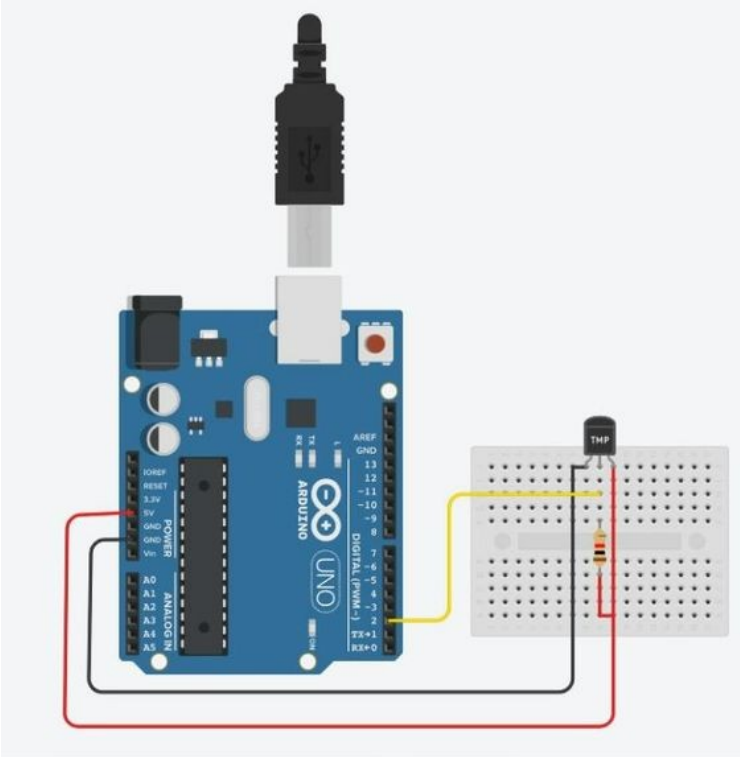


Fig 3.3 — Temperature sensor wiring to Arduino Uno

3.2.3 SONAR — Arduino Connection

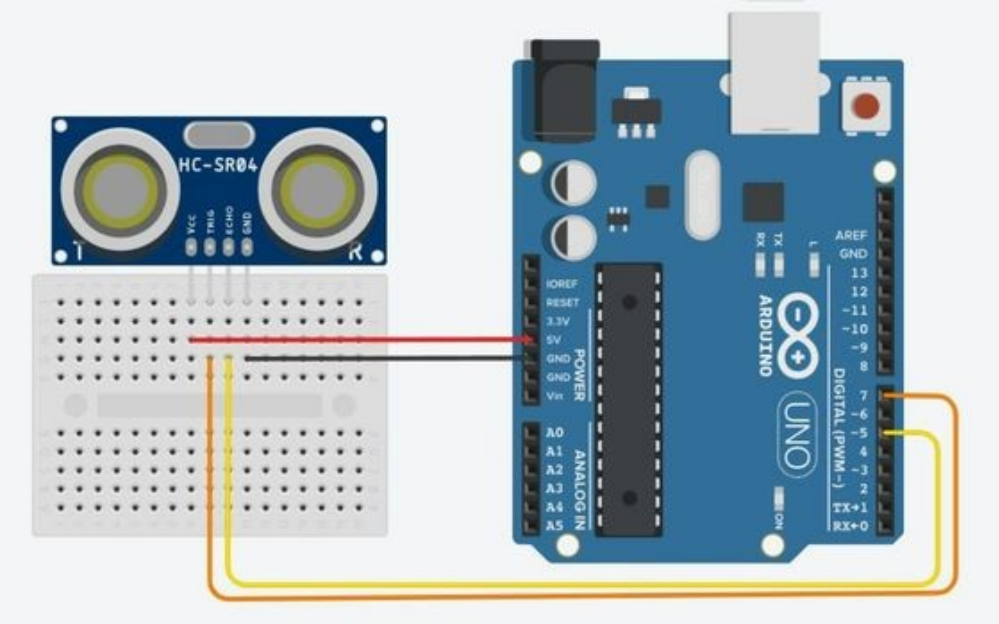


Fig 3.4 — HC-SR04 SONAR wiring to Arduino Uno

3.2.4 pH Sensor — Arduino Connection

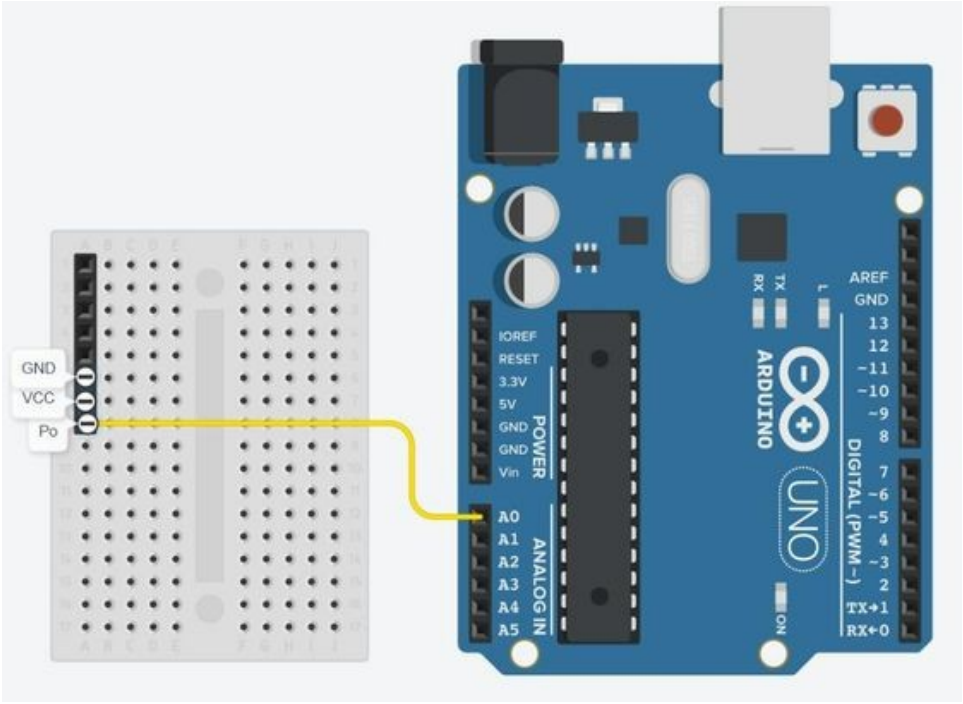


Fig 3.5 — pH sensor wiring to Arduino Uno

3.3 Functionality Analysis

3.3.1 Sensor Data Display on OLED

The system gathers sensor readings — pH, temperature, and water level — and displays them on an OLED screen in real time. This allows users to monitor crucial aquarium parameters conveniently and promptly.

3.3.2 pH Sensor & GSM Integration

Trigger condition: pH > 10. When detected, the GSM module sends an immediate SMS alert to the registered mobile number. The alert message reads: **"Boss I'm dying."** The boolean flag **msg_sent_ph** prevents duplicate messages until pH returns to safe levels (≤ 5).

3.3.3 User-Initiated Water Change via GSM & Pump Control

Upon receiving the character **'m'** via SMS, the Arduino activates motor1 (inlet pump) to HIGH, initiating a water change. The outlet pump is activated automatically when the SONAR detects that water distance ≥ 12 cm (low level).

3.3.4 Heater Activation & Temperature Monitoring

Trigger condition: Water temperature < 25°C. The Arduino sets TEMP_ALERT_PIN (pin 8) HIGH, activating the heater and simultaneously illuminating the LED indicator. When temperature returns to ≥ 25°C, the pin is set LOW.

3.3.5 Automated Feeding with Servo Motor

After every **12-hour interval** (43,200,000 ms delay), the Arduino triggers the servo motor to rotate to **45°** (dispensing food), then returns to **0°** after 1 second. This provides consistent scheduled nourishment without manual intervention.

4. Features

4.1 Automation Highlights

- **Intelligent Sensor Integration:** pH, temperature, and water-level sensors feed real-time data to the Arduino, which uses conditional thresholds to trigger actions — heater, pumps, or GSM alerts — automatically.
- **Precise Servo Motor Control:** Timed loops within the Arduino code activate the servo motor at predefined 12-hour intervals, dispensing fish feed with precision.

4.2 Advanced Monitoring Techniques

- **Real-Time OLED Visualization:** Sensor data is formatted and pushed to the OLED display every loop cycle, providing an instant visual readout of pH, water level, and temperature.

- **GSM Remote Monitoring & Alerts:** Conditional statements in the Arduino code detect threshold breaches and dispatch SMS notifications via the GSM library, ensuring the user is informed regardless of physical proximity.

4.3 Algorithmic Control Precision

- **Conditional Logic:** if-else statements interpret sensor readings against thresholds, triggering heater activation, pump operation, or GSM alerts as required.
- **Timed Sequences:** delay() and loop timing functions govern the 12-hour feeding schedule, ensuring predictable and consistent automation.

5. Implementation

5.1 Hardware Implementation

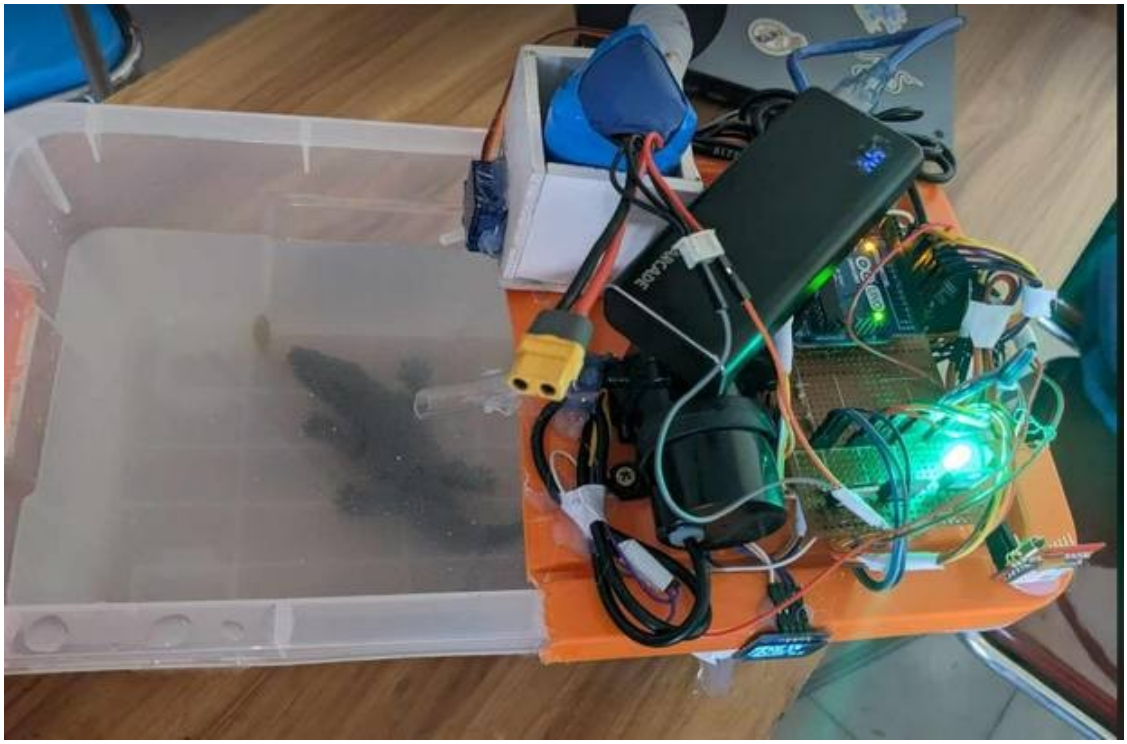


Fig 5.1 — Complete assembled automated aquarium system

5.2.1 Sensors Integration

- **pH Sensor:** Connected to Arduino analog pin A0 with voltage divider. Outputs voltage proportional to solution acidity, interpreted by the calibration formula: pH = −5.70 × V + 34.50
- **Temperature Sensor:** Interfaced via digital pin 2 using OneWire protocol (DallasTemperature library). Transmits digital temperature values.
- **Water Level (SONAR):** Trigger on digital pin 7, Echo on digital pin 5. Uses pulseIn() to measure ultrasonic round-trip time and compute distance in cm.

5.2.2 Actuators and Modules

- **Heater:** Connected to digital pin 8 via relay/transistor. Activated HIGH when temperature < 25°C.
- **Servo Motor:** Connected to PWM pin 11. Moves to 45° to dispense feed, returns to 0° after dispensing.
- **GSM Module:** Software serial on pins 9 (TX) and 10 (RX). AT command protocol for SMS send/receive.
- **OLED Display:** I2C interface on analog pins A4 (SDA) and A5 (SCL). Uses Adafruit_SSD1306 library at address 0x3C.
- **Water Pump:** Motor1 on digital pin 6, Motor2 on digital pin 3. Activated based on SONAR distance readings and remote GSM commands.

5.3 Arduino Pin Connections

5.3.1 SONAR (HC-SR04) → Arduino

SONAR Pin	Arduino Pin
GND	GND
Echo	Digital 5
Trig	Digital 7
Vcc	5V

5.3.2 pH Sensor → Arduino

Sensor Pin	Arduino Pin
V+	5V
GND	GND
P0	Analog A0

5.3.3 OLED Display → Arduino

OLED Pin	Arduino Pin
GND	GND
Vcc	5V
SCL	Analog A5
SDA	Analog A4

5.3.4 Temperature Sensor → Arduino

Sensor Wire	Arduino Pin
Yellow (Data)	Digital 2
Red	5V
Black	GND

5.3.5 GSM Module → Arduino

GSM Pin	Arduino Pin
Vcc	5V
GND	GND

Tx	Digital 9
Rx	Digital 10

5.3.6 Servo Motor → Arduino

Motor Pin	Arduino Pin
Signal	Digital 11 (PWM)
Vcc	5V
GND	GND

6. Evaluation of the Automated Aquarium System

6.1 Performance Analysis

The performance analysis reveals commendable reliability in maintaining the aquarium's environmental parameters. Critical evaluation highlights occasional latency in sensor responsiveness during rapid environmental fluctuations. Minor inaccuracies in certain sensor readings demand fine-tuning or calibration to optimize system responsiveness and accuracy.

6.2 Impact

The system has significantly reduced manual workload associated with monitoring and adjusting aquarium parameters. Automated functionalities have led to a discernible improvement in water quality, promoting a healthier ecosystem. GSM integration empowers users with timely alerts, enabling swift intervention to preserve aquarium stability.

6.3 User Experience

Users appreciate the intuitive OLED display providing real-time insights into crucial parameters. Automated feeding via the servo motor has received overwhelmingly positive feedback. The GSM remote accessibility has been lauded, though the interface's simplicity warrants enhancement. There is a growing desire for an expanded user interface allowing greater customization.

7. Future Enhancements

7.1 Integration of Oxygen Sensor

- Addition of a dissolved oxygen sensor for real-time monitoring of O_2 levels.
- Enables precise adjustments to maintain optimal oxygen levels crucial for aquatic life.
- Strengthens the system's capability to sustain a healthy aquatic environment.

7.2 Implementation of AI Technology

- Incorporation of AI for predictive analysis and adaptive control.
- AI-driven learning from data patterns to predict changes in aquarium parameters.
- Facilitates preemptive adjustments, optimizing the environment proactively.
- Evolves the system into an intelligent, self-regulating platform for enhanced management.

8. Challenges

Challenge	Description
Synchronization Delays	Delays between sensor readings and actuator responses caused intermittent disruptions, demanding fine-tuning for real-time responsiveness.
GSM Module Coding	Ensuring seamless data transmission and reception required meticulous AT command development and testing for reliable remote communication.
Water Level Sensor Issues	Initial setbacks with traditional water level sensors prompted adoption of SONAR technology for accurate and consistent measurements.
Code Optimization	Sluggish code execution led to system inefficiencies; dedicated efforts were invested in optimizing code for faster and streamlined performance.
Code Syntax Errors	Frequent syntax errors during development demanded meticulous debugging and thorough validation to ensure smooth execution.
Module Integration Debugging	Integrating diverse modules demanded rigorous debugging to harmonize functionalities and ensure seamless coordinated operation.

9. Estimated Cost

Item	Cost (BDT)
Push Button	20
Arduino Uno R3	1,100
Vero Board	30
Resistor	10
Bread Board	160
OLED Display	700
Jumper Wire	100
GSM Module	2,500
Servo Motor	1,500
Pump	450

Heater	500
Temperature Sensor	800
pH Sensor	2,500
TOTAL	**10,370 BDT**

10. Conclusion

As this aquarium project concludes, it has been an exhilarating journey merging cutting-edge technology with the marvels of nature. Sensors and Arduinos work together underwater, orchestrating a synchronized ballet of data and commands. The project demonstrates how sophisticated IoT systems can intricately coexist with, and enhance, the natural environment within an aquarium setting.

Fish tanks are no longer merely glass boxes — they are intricate ecosystems designed for the ultimate comfort and joy of aquatic life. This fusion of tech and nature, where innovation and nature's brilliance collide, paves the way for the most extraordinary automated aquatic realms.

Digital Alarm Clock

A Project Report

---

1. Objective

The objective of this project is to design and implement a Digital Alarm Clock that not only displays time and alarms but also incorporates interactive and engaging features such as multi-alarm support, customizable snooze durations, and an innovative alarm deactivation system to ensure active user participation when dismissing alarms.

1.1 Goals

The goal is to create a user-friendly digital alarm clock that:

1. **Multi-Alarm Management:** Allows users to set and manage multiple alarms easily through the keypad.
2. **Flexible Snooze:** Provides customizable snooze options (1, 2, or 3 minutes) based on user preference.
3. **Interactive Dismissal:** Enhances alarm dismissal by requiring users to input a secret key, ensuring the user is awake and alert.
4. **Full Peripheral Integration:** Integrates a loud buzzer, a water-spray motor, a clear LCD display, and a responsive 4×4 keypad for effective interaction.

2. Required Components

Component	Cost (BDT)
Microcontroller (AT89C52)	5,000
Keypad (4×4)	Built-in
Jumper Wires	200
Buzzer	Built-in
LED	Built-in
LCD (LM016L)	Built-in
Motor (water pump)	1,500
MOSFET	50
TOTAL	**6,750 BDT**

3. Circuit Diagram

3.1 Proteus Simulation Layout

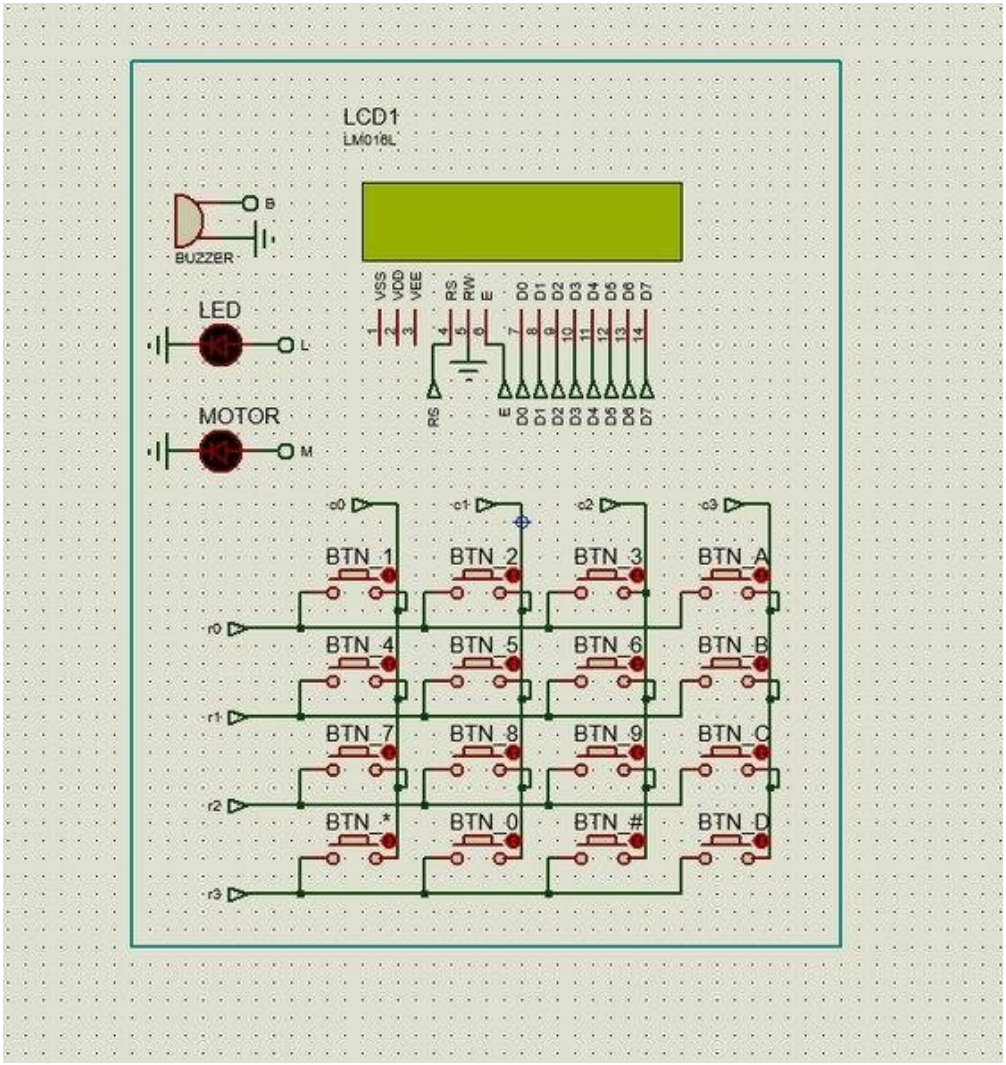


Fig 3.1 — Proteus simulation: LCD, Buzzer, LED, Motor, and 4×4 Keypad connected to AT89C52

3.2 Schematic Diagram

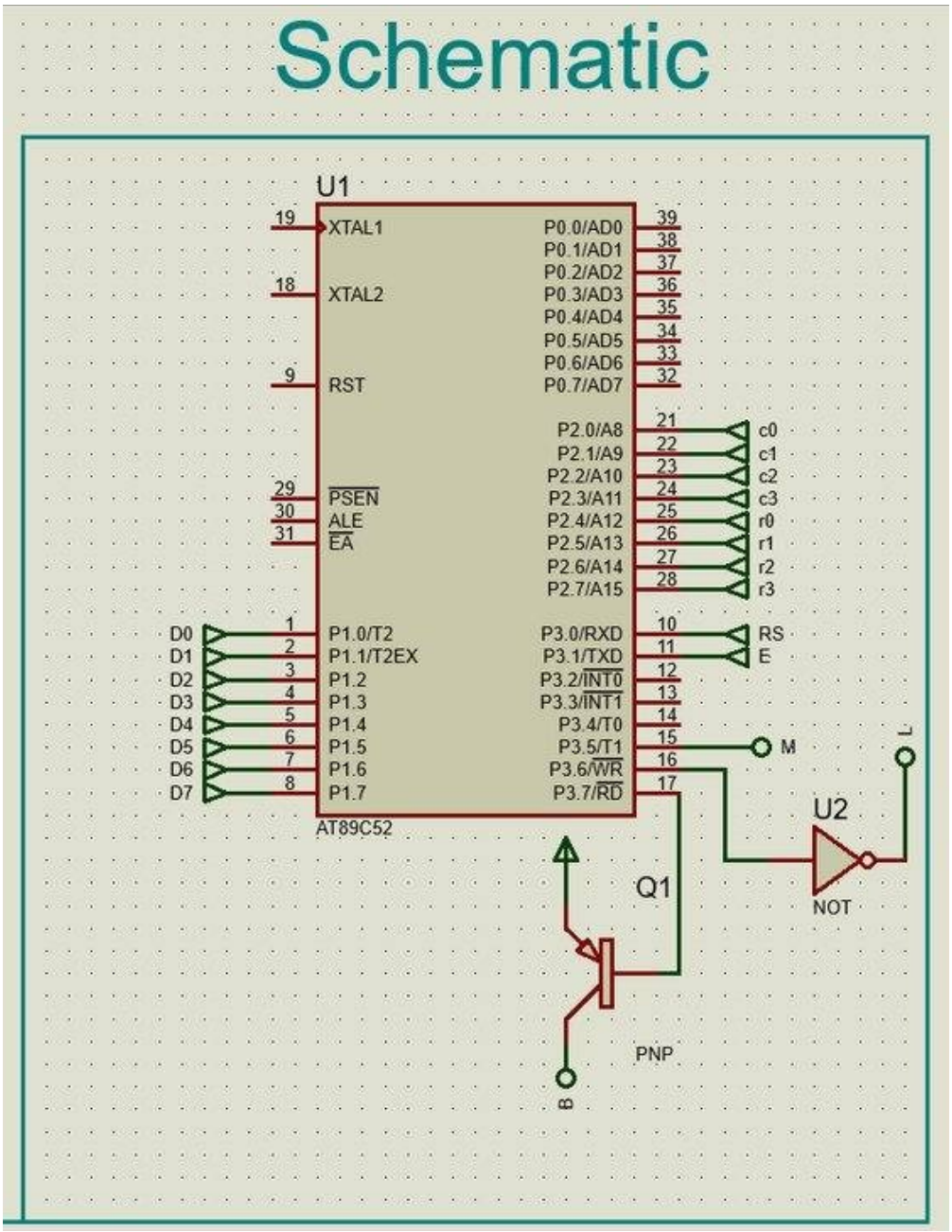


Fig 3.2 — Full schematic: AT89C52 microcontroller pin assignments, PNP transistor motor driver, and NOT gate

3.3 Port & Pin Assignment

Port / Pin	Connected To	Description
Port 1 (P1.0–P1.7)	**LCD (LM016L)**	Data bus — sends 8-bit data/commands to the LCD
Port 2 (P2.0–P2.3)	**Keypad Rows (r0–r3)**	Lower nibble used for keypad row scanning
Port 2 (P2.4–P2.7)	**Keypad Cols (c0–c3)**	Upper nibble used for keypad column scanning

P3.0 (RS)	**LCD RS pin**	Register Select — distinguishes command vs data
P3.1 (Enable)	**LCD Enable pin**	Strobe pin — latches data into LCD on falling edge
P3.5	**Motor (via MOSFET)**	Water-spray motor — activated in Alarm Resolve Mode
P3.6	**Alarm Indicator LED**	Illuminates when alarm is active (ALRMSET)
P3.7	**Buzzer**	Sounds audible alert in Alarm Resolve Mode

4. Features

4.1 Clock Display Mode (Mode 1 — Default)

- The clock continuously displays current time (HH:MM:SS) on the LCD, updating every second.
- The display remains active at all times, ensuring the user can view the current time at a glance.
- When an alarm is set, an **'A'** indicator appears in the top-right corner of the LCD.

4.2 Edit Time Mode (Mode 2 — triggered by 'D' key)

- Allows the user to manually set hours, minutes, and seconds via the keypad.
- Validates all inputs (H < 24, M < 60, S < 60) before accepting.
- After saving, the mode automatically returns to Clock Display Mode.

4.3 Alarm Set Mode (Mode 3 — triggered by '*' key)

- Allows the user to set an alarm by entering hours and minutes (HH:MM).
- Validates inputs (H < 24, M < 60) and confirms with a 0/1 flag.
- After setting, returns to Clock Display Mode with the 'A' indicator active.

4.4 Alarm Resolve Mode (Mode 4 — auto-triggered on alarm match)

- **Buzzer:** Activates the buzzer (P3.7 set LOW) to produce a loud auditory alert.
- **Motor:** Activates the water-spray motor (P3.5 HIGH) via a MOSFET/PNP driver, providing a physical stimulus.
- **Secret Key:** Prompts the user to enter a secret key on the LCD. If incorrect, the alarm continues ringing.
- **Stop / Snooze:** On correct key entry, the user presses 0 to stop the alarm, or 1/2/3 to snooze for 1, 2, or 3 minutes respectively.

5. System Flowchart

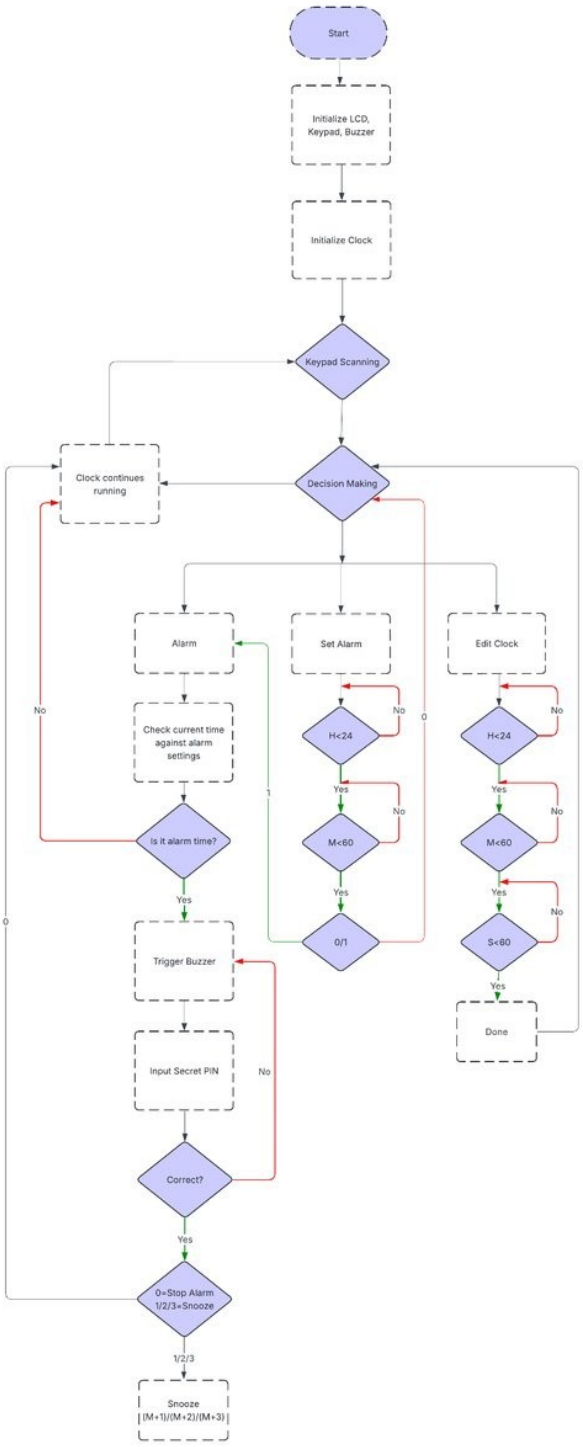


Fig 5.1 — Complete system flowchart: initialization → keypad scan → decision → Clock / Set Alarm / Edit Clock / Alarm Resolve branches

The flowchart illustrates the four operating branches. After initialization, the system enters a continuous keypad scanning loop. The Decision Making node routes execution to the

appropriate mode. The Alarm branch checks the current time against alarm settings each loop; when matched, Trigger Buzzer is called and the user must enter the secret key before reaching the Stop/Snooze decision.

6. Working Principle

6.1 Initialization Phase

The program begins by configuring all peripherals and zeroing all time variables:

Module	Initialization Action
LCD	Configured with 38H (8-bit, 2-line), 0CH (display on), 06H (entry mode), 01H (clear)
Keypad	PORT 2 set to 0FFH (all lines HIGH, ready for scanning)
Timer 1	Set in Mode 1 (16-bit), reload values 0DBH/0F0H for 10 ms interrupt intervals
Variables	REALHOUR, REALMINT, REALSCND, ALRMHOUR, ALRMMINT, ISALRM, MODE all set to 0
Buzzer	SETB P3.7 (HIGH = off, active-low configuration)
Motor	CLR P3.5 (LOW = off)

6.2 Main Loop Operation

The code enters an infinite loop (MAINLOOP) performing three continuous tasks:

5. Scans the keypad for any user input and stores the detected key in KEYMEM.
6. Updates the LCD display with the current time (REALHOUR:REALMINT:REALSCND).
7. Compares **REALMINT / REALHOUR** against **ALRMMINT / ALRMHOUR** — if both match and ISALRM ≠ 0, MODE is set to 04H (Alarm Resolve).

6.3 Mode Switching

Mode	Name	Trigger	Description
01H	**Clock Display**	Default / on save	Shows HH:MM:SS in real time. 'A' shown if alarm active.
02H	**Edit Time**	'D' key pressed	User enters H, M, S. Validates ranges. Returns to Mode 1.
03H	**Alarm Set**	'*' key pressed	User enters alarm HH:MM and enables (0/1). Returns to Mode 1.
04H	**Alarm Resolve**	Clock matches alarm time	Buzzer + motor active. User must enter secret key, then

			stop/snooze.

6.4 Timer 1 Interrupt (TIMER1ISR) — 10 ms intervals

Timer 1 is configured to interrupt every 10 ms. On each interrupt, the following cascade of counter updates takes place:

Counter	Overflow Condition	Action on Overflow
REALMSCND	Reaches 100 (= 1 second)	Reset to 0; increment REALSCND by 1
REALSCND	Reaches 60 (= 1 minute)	Reset to 0; increment REALMINT by 1
REALMINT	Reaches 60 (= 1 hour)	Reset to 0; increment REALHOUR by 1
REALHOUR	Reaches 24 (= 1 day)	Reset to 0 (24-hour wrap-around)

The Timer 1 reload values are **TH1 = 0DCH, TL1 = 000H** on each re-entry, maintaining precise 10 ms intervals. While in Alarm Resolve Mode (MODE = 04H), the ISR also keeps the buzzer active (**CLR BUZZER**) during each tick.

6.5 Keypad Scanning & Input Handling

The 4×4 keypad is scanned using a row-column method on **Port 2**. Columns are driven HIGH (0F0H) first to detect active rows, then rows are driven HIGH (00FH) to detect active columns:

- **Column scan:** PORT2 ← 0F0H, read upper nibble → call KEYPAD_IDX → store column index in R0
- **Row scan:** PORT2 ← 00FH, read lower nibble → call KEYPAD_IDX → store row index in R1
- **Key lookup:** Index = R0×4 + R1, look up KEYPADBUF table → store ASCII char in KEYMEM
- **Debounce:** KEY_TAKE_ONE_DIGIT scans twice with a DELAY between reads and only accepts stable, consistent key presses

Key	Action
0–9	Digit input for time, alarm, snooze, and secret key entry
*** (star)**	Switch to Mode 3 (Alarm Set Mode)
D	Switch to Mode 2 (Edit Time Mode)
0 (in alarm)	Stop alarm — clears ISALRM and ISSNZ
1/2/3 (in alarm)	Snooze for 1/2/3 minutes — adds to ALRMMINT

7. Hardware Implementation

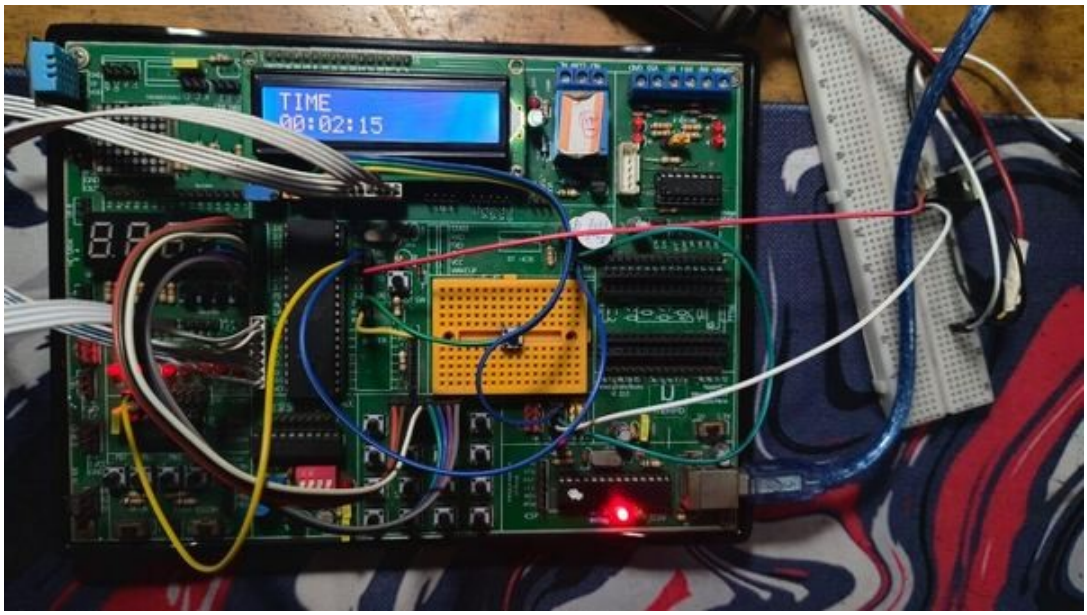


Fig 7.1 — Physical implementation on AT89C52 board showing LCD displaying TIME 00:02:15, keypad, and alarm indicator LED (red)

The system was implemented on a dedicated AT89C52 development board. The LCD clearly displays the clock in TIME HH:MM:SS format. The 4×4 matrix keypad is visible at the bottom. The red LED (alarm indicator) is visible at the right. The motor driver (PNP + MOSFET) is connected to P3.5 for the water-spray mechanism.

8. Problems Faced

8.1 Software Issues

Problem	Description
Concurrent Input & Timer	Difficult to accept keypad input for Alarm Set Mode while Timer 1 was running and continuously refreshing the LCD display.
LCD Refresh Glitch	Real-time display was not updating as expected — residual/garbage characters appeared on the LCD.
Keypad Debouncing	Keypad produced double inputs due to contact bounce; resolved using double-scan debouncing in KEY_TAKE_ONE_DIGIT.
Multiple Alarms	Second alarm was not triggering properly — implementing multiple alarms would require enhanced memory management and interrupt prioritization.

LED Puzzle Complexity	The LED-pattern puzzle for dismissal was too complex to implement reliably; replaced with a simpler secret key input.

8.2 Hardware Issues

Problem	Description
Insufficient Motor Power	The microcontroller board could not supply enough current on P3.5 to drive the motor; a MOSFET/PNP driver stage was required.
Faulty Port 1	Port 1 of the board was defective; Port 0 was used instead for the LCD data bus in the physical build.
Loose Burning Switch	The programming (burning) switch had a loose connection, causing multiple failed code-upload attempts before a successful flash.

9. Conclusion

9.1 Delivered Outcomes

8. **Real-Time Clock:** Accurately tracks and displays HH:MM:SS using Timer 1 interrupts. Users can manually edit the time via the keypad.
9. **Alarm System:** Triggers both auditory (buzzer) and physical (water-spray motor) stimuli for effective wake-up alerts. Snooze of 1, 2, or 3 minutes is supported.
10. **Interactive Dismissal:** The secret key input ensures user engagement. The alarm continues until the correct key is provided.

9.2 Areas for Improvement

- **Multiple Alarms:** Requires enhanced memory management and interrupt prioritization beyond the current single-alarm implementation.
- **Debounce Circuit:** A dedicated hardware debounce circuit (RC filter or Schmitt trigger) would further stabilize keypad inputs.
- **LED Puzzle Feature:** Restoring the original LED-pattern dismissal challenge would significantly increase user engagement.
- **Motor Power:** An external motor driver IC (e.g. L293D) would ensure reliable motor operation independent of the microcontroller's current capacity.

This project demonstrated the value of iterative testing, flexibility in problem-solving, and striking a balance between technical viability and ambition. It provided practical experience with peripheral interfacing, microcontroller programming in Assembly, and debugging under real-world hardware constraints.

High Voltage Taser

A Project Report

---

1. Introduction

A Taser, or stun gun, involves power electronics in its design and operation. Power electronics is a field of electrical engineering that deals with the control and conversion of electrical power. In the case of a Taser, power electronics is used to generate, control, and deliver high-voltage, low-current pulses.

For the EEE 4504 project, a prototype Taser was constructed that takes **3.7V DC** as input and delivers an output in the **kilovolt range**, demonstrated through visible electric sparks.

2. Theoretical Background and Methodology

The approach taken to build the Taser is based on a Joule Thief circuit combined with a Cockcroft-Walton voltage multiplier. Together, these two stages step up a 3.7V battery input to an estimated output of ~8kV.

Stage	Circuit	Function
Stage 1	**Joule Thief**	3.7V DC → ~800V AC
Stage 2	**Cockcroft-Walton (3×)**	~800V AC → ~8kV DC

2.1 Joule Thief Circuit

The Joule Thief is a simple voltage booster circuit that efficiently extracts energy from a low-voltage source — in this case a 3.7V DC battery — to power a higher-voltage load. The basic components are an inductor (coil), a transistor (2N2222), a resistor (1.5kΩ), and a feedback winding.

Principle

The Joule Thief operates on the principle of energy storage and release. The inductor stores energy when the transistor is in the ON state, then releases that stored energy when the transistor turns OFF. The rapid collapse of the magnetic field induces a much higher voltage across the coil.

Operation

1. **Transistor ON:** Current flows through the inductor (primary winding of T1), storing energy in the magnetic field.

2. **Transistor OFF:** The magnetic field collapses. The induced voltage across the coil spikes to ~800V AC, which is fed into the voltage multiplier stage.

Component	Value / Part	Role
Transistor Q1	2N2222	Switching element — rapidly turns on/off to collapse inductor field
Resistor R1	1.5 kΩ	Base bias resistor — limits base current to transistor
Transformer T1	20:30:500	Step-up transformer — feedback winding (20t primary, 30t feedback, 500t secondary)
Capacitor C1	0.1 µF	Input decoupling capacitor

2.2 Cockcroft-Walton Voltage Multiplier

A voltage multiplier circuit further increases the output voltage obtained from the Joule Thief. Voltage multipliers use capacitors and diodes to combine multiple voltage pulses, resulting in an output voltage that is a multiple of the input voltage. The Cockcroft-Walton multiplier was chosen for this project.

Structure

The Cockcroft-Walton multiplier consists of a series of diodes and capacitors arranged in a ladder-like structure. For **n cascades**, the circuit multiplies the input voltage by **2n**. This project uses **3 cascades**, yielding a theoretical multiplication of ×6 on top of the transformer output.

Operation

3. **Positive half-cycle:** Odd-column capacitors (C3, C5, C7) charge through the diodes.
4. **Negative half-cycle:** Even-column capacitors (C2, C4, C6) charge by stacking the previously charged capacitors in series.
5. **Cumulative voltage:** Each stage adds to the total, resulting in an output voltage across the final capacitor that is many times higher than the input.

Component	Spec	Role
Diodes D1–D6	1N4007	Rectification — allow current to flow only in one direction per stage
Capacitors C2–C6	103J / 2kV	Ceramic capacitors rated 2kV — store charge at each stage of the ladder
Capacitor C7	27pF / 2kV	Output filter capacitor — smooths the final high-voltage DC output
Capacitor C1	223J / 2kV	Additional 2kV capacitor for stability at the multiplier input

2.3 Combining Joule Thief with Voltage Multiplier

In the Taser, the Joule Thief circuit generates a series of high-voltage AC pulses (~800V) from the 3.7V battery. The Cockcroft-Walton multiplier then receives these pulses and further increases the voltage. The final high-voltage output (~8kV) is delivered at the output terminals, producing visible electric sparks.

3. Circuit Diagram

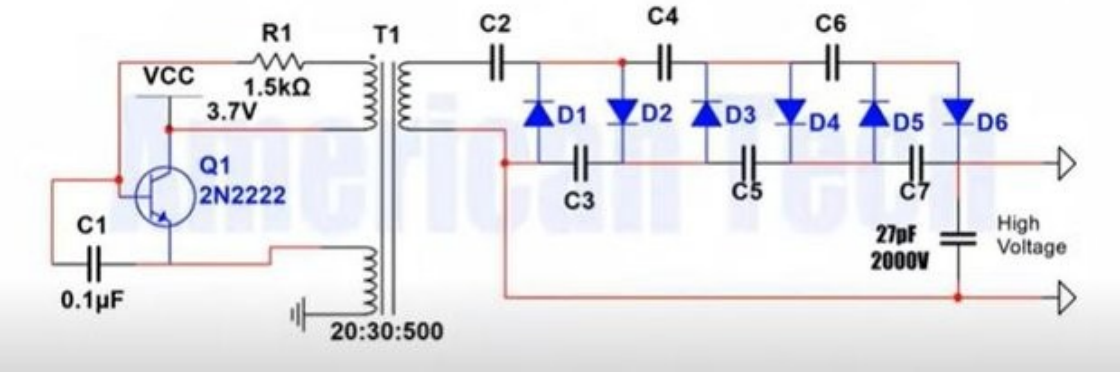


Fig 3.1 — Complete Taser circuit: Joule Thief (left) + Cockcroft-Walton 3-stage multiplier (right)

The left half of the circuit (VCC 3.7V, R1, Q1, T1, C1) forms the Joule Thief oscillator. The right half (D1–D6, C2–C7) forms the three-stage Cockcroft-Walton ladder, with the High Voltage output at the far right. The transformer T1 has a turns ratio of 20:30:500.

4. Analysis

After providing 3.7V DC as the input to the Joule Thief stage:

Measurement Point	Observed / Calculated Value
Input Voltage	3.7V DC (single cell Li-ion battery)
Transformer Output (T1)	**~800V AC**
Multiplier Cascades	3 stages (×6 multiplication factor)
Estimated Output Voltage	**~8,000V DC (8 kV) — not directly measurable**
Measurement Attempt	Multimeter burned out when attempting to measure output voltage
Observable Output	**Visible electric sparks at output terminals**

5. Hardware Implementation

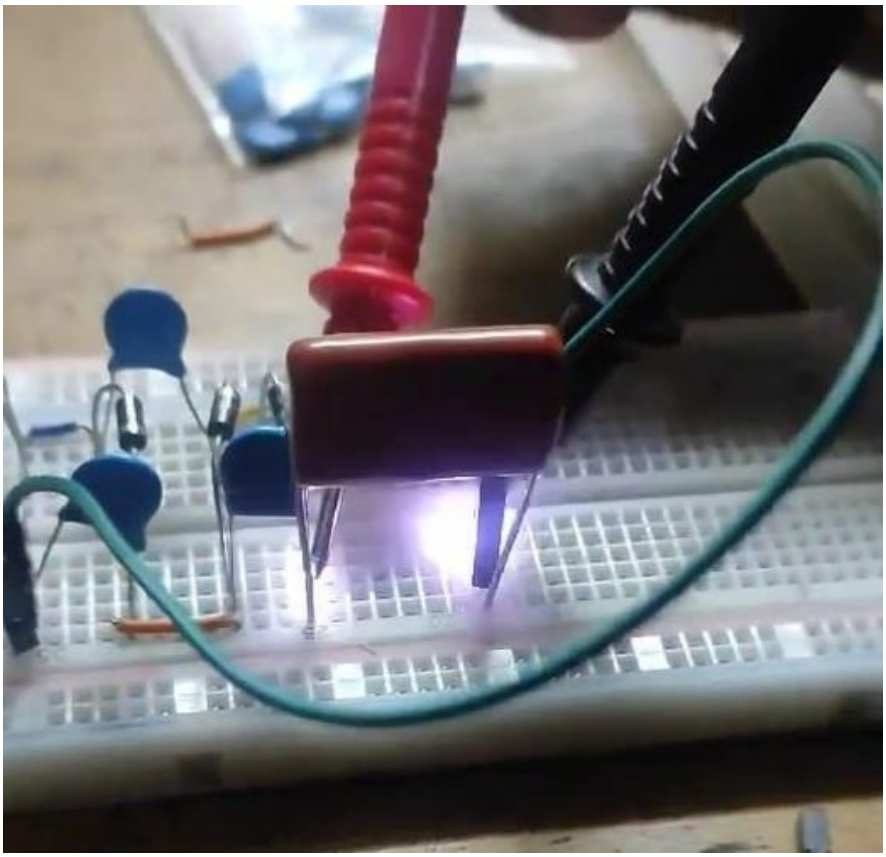


Fig 5.1 — Prototype breadboard implementation showing visible high-voltage sparks at the output

The circuit was assembled on a breadboard. The visible spark discharge between the output terminals confirms the generation of high-voltage output (~8kV). The blue ceramic capacitors (2kV rated) and the hand-wound transformer (T1 with 20:30:500 turns ratio) are clearly visible on the board.

6. Equipment and Cost

Component	Part No. / Spec	Qty	Cost (BDT)
Diode	1N4007	6	30
Ceramic Capacitor	103J 2kV	4	16
High Frequency Transformer	20:30:500	1	200
Capacitor	223J 2kV	1	20
Resistor	10 kΩ	1	2
TOTAL			**268 BDT**

7. Project Outcome

- **Self-Defence Prototype:** Gained practical experience in constructing a high-voltage stun device, relevant to personal safety applications.
- **High Voltage Safety:** Developed an understanding of the safety protocols required when working with voltages in the kilovolt range.
- **Joule Thief Circuit:** Achieved thorough understanding of the Joule Thief oscillator — how energy is stored in an inductor and released at a higher voltage.
- **Cockcroft-Walton Multiplier:** Gained understanding of how cascaded diode-capacitor ladders multiply voltage with each successive stage.
- **High Voltage Observation:** Observed visible spark discharge as direct evidence of the ~8kV output, confirming circuit functionality.

8. Discussion

The EEE 4504 project explored high-voltage generation using readily available components. The core concept — stepping up **3.7V** to an estimated **~8kV** — was achieved by coupling a Joule Thief oscillator with a Cockcroft-Walton multiplier.

The project's significance extends beyond its electrifying output. Invaluable hands-on experience was gained in power electronics, demystifying the workings of both circuit topologies. The project also underscored the paramount importance of high-voltage safety protocols — a crucial lesson for anyone working in this domain.

Limitations

- **Measurement challenge:** The actual output voltage could not be measured precisely — the standard multimeter was destroyed when measurement was attempted, confirming the voltage was well beyond its rated range.
- **Breadboard limitations:** Breadboard construction is not ideal for high-voltage circuits due to parasitic capacitances and insufficient creepage distances between conductors.

Future Directions

- **Optimization:** Efficiency and miniaturization improvements for a more compact, portable form factor.
- **PCB implementation:** Proper PCB layout with adequate creepage and clearance distances for high-voltage safety.
- **Applications:** Adaptation of these voltage-multiplication techniques for pest control devices, medical equipment, or plasma generation.
- **Responsible development:** Emphasizing ethical considerations and strict safety standards when further developing high-voltage technology.